Classification Hiérarchique de Textes utilisant des Méthodes à Noyau pour le Traitement Automatique du Langage Naturel

Fatou Kiné TOURE (fatou.k.toure@aims-senegal.org)
African Institute for Mathematical Sciences (AIMS), Senegal
Supervised by: Doctor Seydou Nourou Sylla
University of Alioune DIOP, Senegal

26 février 2026

Submitted in partial fulfilment of the requirements for the award of Master of Science in Mathematical Sciences at AIMS Senegal

DECLARATION

Je déclare sur l'honneur que ce mémoire, intitulé Classification Hiérarchique de Textes utilisant des Méthodes à Noyau pour le Traitement Automatique du Langage Naturel, est le fruit de mon travail personnel, réalisé sous la direction de Dr Nourou Sylla. Toutes les sources utilisées, qu'il s'agisse d'ouvrages, d'articles scientifiques ou d'autres ressources, ont été dûment citées et référencées. Ce travail n'a pas été soumis auparavant pour l'obtention d'un autre diplôme.

Student : Fatou Kiné TOURE

Supervisor : Doctor Seydou Nourou Sylla (scan) supervisor's signature

tutor : Joel Rostand Nteupe (scan) tutor's signature

Remerciements

Je rends grâce à Dieu pour la force et l'inspiration qu'Il m'a offertes tout au long de ce mémoire.

Mes profonds remerciements vont à mon encadreur, Docteur Seydou Nourou Sylla, pour son accompagnement rigoureux et ses conseils précieux. Je remercie également mon tuteur,Joel Rostand Nteupe, pour ses orientations pertinentes et son soutien constant. Un merci spécial à Ilimane Gning, mon camarade promotion régulard de 2025, pour son aide inestimable et nos discussions enrichissantes.

À mes compagnons de guerre, Malick Ba, ma jumelle Seynabou Diagne, Mamie Sambou,Omar Gueye merci pour votre amitié fidèle, vos encouragements sans faille et pour m'avoir soutenu dans les moments les plus ardus.

À mes sœurs, Seynabou Toure, Ngone Toure, Khoudia Toure, ma profonde gratitude pour votre amour inconditionnel, vos mots réconfortants et votre soutien moral, véritables piliers de cette aventure.

À mes camarades promotion de 2025, Ousmane Tienta, Pontiane, Abou Birane Baro, ma colocataire Brenda, Magatte, Ndeye Bineta, Anna Niane, Amy Collé Diop, Rokhaya, Abdou Latifou, Cheikh Fall, ainsi qu'à tous nos amis réguliers, en présentiel comme en ligne, et à toutes les coopératives, un immense merci pour votre énergie, votre accompagnement constant et les moments partagés qui ont marqué mon parcours académique.

À mes chers parents, Anna Camara et Mamadou Touré, merci pour votre soutien indéfectible et votre amour sans limite, qui m'ont portée à chaque étape. À ma seconde maman, Khady Ndiaye, qui m'a élevée depuis mon plus jeune âge et a façonné la personne que je suis aujourd'hui, ma gratitude est infinie. À la famille Ndaw et à tous les membres de ma famille, de près ou de loin, je vous remercie du fond du cœur pour votre présence précieuse et vos encouragements, qui ont gravé des souvenirs inoubliables dans ce parcours.

Enfin, à toutes les personnes non citées qui ont contribué, de près ou de loin, à ce mémoire, je vous adresse mes sincères remerciements pour votre soutien et votre générosité, qui resteront à jamais gravés dans mon cœur.

DEDICATION

Je dédie ce mémoire :

À mes parents, Anna Camara et Mamadou Touré,
pour leur amour incommensurable, leur confiance sans faille et leur soutien constant à chaque étape de mon parcours.

À ma seconde maman, Khady Ndiaye,
qui a toujours été à mes côtés, m'a guidée et inspirée, façonnant la personne que je suis aujourd'hui.

À mes sœurs, Seynabou, Ngone et Khoudia Toure,
pour leur complicité unique, leurs encouragements et leur présence rassurante, qui m'ont porté même dans les moments les plus difficiles.

Votre amour et votre soutien sont le socle de cette réussite.

Abbreviations and Notations

Liste des Abréviations et Notations

Abréviations

- **GPA** : Grade Point Average (Moyenne pondérée des notes)
- **YL1** : Niveau parent
- **YL2** : Niveau intermédiaire
- **Y** : Niveau feuille / spécialité
- **BERT** : Bidirectional Encoder Representations from Transformers
- **CHT** : Classification Hiérarchique de Textes
- **CPU** : Central Processing Unit
- **ELMo** : Embeddings from Language Models
- **FN** : Faux Négatifs
- **FP** : Faux Positifs
- **GPU** : Graphics Processing Unit
- **GPT** : Generative Pre-trained Transformer
- **LCA** : Plus Petit Ancêtre Commun (Lowest Common Ancestor)
- **LCL** : Classifieur Local par Niveau (Local Classifier per Level)
- **LCN** : Classifieur Local par Nœud (Local Classifier per Node)
- **LCPN** : Classifieur Local par Nœud Parent (Local Classifier per Parent Node)
- **OS** : Operating System
- **RAM** : Random Access Memory
- **RBF** : Radial Basis Function
- **RCV1-v2** : Reuters Corpus Volume I - version 2
- **SVM** : Machines à Vecteurs de Support (Support Vector Machines)
- **TALN** : Traitement Automatique du Langage Naturel
- **TF** : Term Frequency (Fréquence des Termes)
- **TF-IDF** : Term Frequency - Inverse Document Frequency
- **VP** : Vrais Positifs
- **WOS** : Web of Science
- **WIPO-alpha** : World Intellectual Property Organization - alpha

Notations Mathématiques

- $A(l)$: Ensemble de tous les ancêtres du label l (y compris l lui-même)
- b : Biais (terme constant de l'hyperplan SVM)

- C : Hyperparamètre de régularisation pour les SVM à marge souple
- $C(n_1, n_2)$: Fonction de similarité de sous-arbres (dans les Tree Kernels)
- d : Degré du polynôme (pour le noyau Polynomial) ; ou dimension de l'espace des embeddings
- D : Corpus de documents
- $\vec{d}$: Vecteur représentant un document (TF-IDF ou agrégation d'embeddings)
- DF(t) : Nombre de documents contenant le terme t
- $f(\mathbf{x})$: Fonction de décision d'une SVM
- γ : Paramètre pour le noyau RBF
- hF1 : Score F1 hiérarchique
- hPrecision : Précision hiérarchique
- hRecall : Rappel hiérarchique
- $K(\mathbf{x}, \mathbf{x}')$: Fonction noyau calculant la similarité entre deux vecteurs $\mathbf{x}$ et $\mathbf{x}'$
- $L(d)$: Ensemble des vrais labels hiérarchiques d'un document d
- n : Nombre de mots dans un document (pour l'agrégation d'embeddings)
- N : Nombre total de documents dans le corpus
- n_1, n_2 : Nœuds dans les arbres T_1 et T_2
- $P(d)$: Ensemble des labels prédits par le modèle pour un document d
- r : Terme de biais (pour le noyau Polynomial)
- sgn$(\cdot)$: Fonction signe
- $\sum_d$: Somme sur tous les documents d
- t : Terme (mot)
- T_1, T_2 : Deux structures arborescentes (pour les Tree Kernels)
- TF(t, d) : Fréquence des termes du terme t dans le document d
- TF-IDF(t, d, D) : Score TF-IDF du terme t dans le document d par rapport au corpus D
- V : Taille du vocabulaire du corpus
- $\mathbf{w}$: Vecteur de poids normal à l'hyperplan de la SVM
- $\vec{w}_i$: Vecteur d'embedding du i-ème mot
- $\vec{w}_i^{\text{contextuel}}$: Vecteur d'embedding contextuel du i-ème mot
- $\mathbf{x}$: Vecteur de caractéristiques d'un point de données
- $\mathbf{x}_i$: Vecteur de caractéristiques du i-ème échantillon
- y_i : Étiquette de classe du i-ème échantillon $(\{-1, 1\})$
- ξ_i : Variable d'écart (slack variable) pour le i-ème échantillon
- f ou $f_p^{(l)}$: classifieur (global ou local pour le parent p au niveau l)
- $\hat{y}_i$ ou $\hat{\mathbf{y}}_i$: prédiction du classifieur pour le document d_i

Résumé

La croissance rapide des publications scientifiques rend indispensable le développement des méthodes automatiques d'organisation de l'information. Ce mémoire s'inscrit dans le domaine du Traitement Automatique du Langage Naturel (TALN) et s'intéresse à la classification hiérarchique de textes (CHT), qui permet d'exploiter les relations sémantiques entre catégories organisées en arbre.

Les expériences sont menées sur le jeu de données *Web of Science*, initialement composé de 11 975 articles, dont 11 913 documents validés après nettoyage et annotés selon une hiérarchie à trois niveaux. L'étude propose une analyse comparative entre la représentation vectorielle TF-IDF et les plongements contextuels issus de BERT, intégrés dans une architecture de type *Local Classifier Per Parent Node (LCPN)*. Des machines à vecteurs de support (SVM) utilisant des noyaux linéaire et RBF sont évaluées face à une baseline de type Forêt Aléatoire.

Les résultats démontrent que l'hybridation du *LinearSVC* avec BERT offre la performance maximale (67,18 % au niveau final Y), surpassant le TF-IDF (65,54 %) tout en confirmant la robustesse des méthodes à noyau. Une découverte majeure de ce travail réside dans le comportement du noyau RBF : alors qu'il est inadapté à la nature creuse du TF-IDF, il devient fonctionnel lorsqu'il traite les vecteurs denses de BERT. L'évaluation met toutefois en évidence que le couple TF-IDF/*LinearSVC* reste la solution la plus efficiente, avec une vitesse d'entraînement 70 fois supérieure à la baseline. Enfin, l'architecture LCPN garantit une cohérence structurelle totale (0,00 % de violation), bien que les limites liées à la propagation des erreurs soulignent l'intérêt de futures méthodes intégrant des contraintes globales.

Mots-clés : Classification hiérarchique de textes ; TALN ; Méthodes à noyau ; SVM ; BERT ; Web of Science.

Abstract

The rapid growth of scientific publications makes the development of automated information organization methods essential. This thesis falls within the field of Natural Language Processing (NLP) and focuses on Hierarchical Text Classification (HTC), which exploits semantic relationships between categories organized in a tree structure.

Experiments were conducted on the *Web of Science* dataset, initially composed of 11,975 articles, of which 11,913 documents were validated after cleaning and annotated according to a three-level hierarchy. The study proposes a comparative analysis between TF-IDF vector representation and contextual embeddings from BERT, integrated into a *Local Classifier Per Parent Node (LCPN)* architecture. Support Vector Machines (SVM) using linear and RBF kernels are evaluated against a Random Forest baseline.

The results demonstrate that the hybridization of *LinearSVC* with BERT offers maximum performance (67.18% at the final level Y), surpassing TF-IDF (65.54%) while confirming the robustness of kernel methods. A major finding of this work lies in the behavior of the RBF kernel : while it is unsuitable for the sparse nature of TF-IDF, it becomes functional when processing dense BERT vectors. However, the evaluation highlights that the TF-IDF/*LinearSVC* pair remains the most efficient solution, with a training speed 70 times faster than the baseline. Finally, the LCPN architecture guarantees total structural consistency (0.00% violation rate), although limitations related to error propagation underscore the potential for future methods integrating global hierarchical constraints.

Keywords : Hierarchical Text Classification ; NLP ; Kernel Methods ; SVM ; BERT ; Web of Science ; LCPN.

Table des matières

Declaration i

Remerciements ii

Dedication iii

Abbreviations and Notations iv

Résumé vi

Abstract vii

1 Introduction 1

1.1 Contexte et problématique 1

1.2 Objectifs du mémoire 1

1.3 Structure du mémoire 2

2 Fondements théoriques du Traitement Automatique du Langage Naturel (TALN) 3

2.1 Introduction au Traitement Automatique du Langage Naturel(TALN) 3

2.1.1 Définition et importance du TALN 3

2.1.2 Évolution et approches principales 3

2.2 Classification de textes 4

2.2.1 Définition et typologie 5

2.2.2 Représentations textuelles 5

3 État de l'art sur la Classification Hiérarchique de Textes (CHT) 6

3.1 Classification hiérarchique de textes (CHT) 6

3.1.1 Modèles locaux et globaux 6

3.1.2 Stratégies top-down et bottom-up 9

3.1.3 Défis spécifiques en CHT 11

3.2 Méthodes à Noyaux (Kernel Methods) en TALN 11
3.2.1 Définition et "Kernel Trick" 12
3.2.2 Noyaux classiques 12
3.2.3 Noyaux spécifiques au TALN 14
3.2.4 Avantages et limitations des méthodes à noyaux 15
3.3 Applications des méthodes à noyaux en Classification Hiérarchique de Textes (CHT) 16
3.3.1 Applications en Médecine 16
3.3.2 Applications en Droit 17
3.3.3 Applications en Classification de Brevets 17
3.4 Comparaison avec approches neuronales 17
3.4.1 Approches neuronales (Deep Learning et Transformers) : Caractéristiques clés 17
3.4.2 Méthodes à Noyaux : Atouts persistants 17
3.4.3 Vers des approches hybrides 18

4 Cadre expérimental et méthodologique de la classification hiérarchique 19
4.1 Présentation du jeu de données 19
4.1.1 Source des données 19
4.1.2 Structure hiérarchique des classes 19
4.1.3 Analyse des biais et limites du jeu de données 20
4.2 Analyse exploratoire des données 21
4.2.1 Répartition des classes 21
4.2.2 Analyse lexicale 22
4.2.3 Outil d'exploration dynamique des données 23
4.3 Prétraitement des textes 24
4.3.1 Prétraitement pour la représentation TF-IDF 24
4.3.2 Prétraitement pour la représentation BERT 25
4.4 Représentation vectorielle 25
4.4.1 Représentation statistique : TF-IDF 25
4.4.2 Représentation contextuelle dense : BERT 25
4.4.3 Comparaison des deux paradigmes 26

4.5 Protocole expérimental comparatif 26
4.6 Impact des représentations sur les méthodes à noyau 26
4.7 Ingénierie des labels et partitionnement 27
4.7.1 Classification plate et hiérarchique 27
4.7.2 Modèles de classification 28
4.7.3 Évaluation des performances 29

5 Résultats et interprétation **31**
5.1 Performances du modèle Baseline (Random Forest) 31
5.1.1 Analyse globale par niveau (Évaluation locale) 31
5.1.2 Analyse granulaire au niveau parent (YL1) 31
5.1.3 Analyse au niveau intermédiaire (YL2) 31
5.1.4 Analyse au niveau feuille (Y) 33
5.1.5 Analyse comparative basée sur le F1-score 35
5.1.6 Discussion comparative des performances entre niveaux 35
5.2 Validation statistique et robustesse du modèle (5-Fold Cross-Validation) 36
5.3 Analyse de la cohérence et de la granularité des prédictions 37
5.4 Évaluation comparative des méthodes à noyau 38
5.4.1 Interprétation de la robustesse sémantique 39
5.5 Comparaison avec les représentations contextuelles (BERT) 39
5.5.1 Analyse de la supériorité du noyau linéaire sur BERT 40
5.5.2 Validation croisée et robustesse 40
5.5.3 Synthèse de la réussite globale et garantie de cohérence 41

Conclusion et Perspectives **43**

Annexe : Protocole de Reproductibilité **44**

References **47**

1. Introduction

1.1 Contexte et problématique

Avec l'augmentation massive des données textuelles disponibles sur le web, dans la littérature scientifique et dans les bases juridiques, la nécessité de développer des méthodes automatiques d'organisation de l'information est devenue cruciale [15, 25]. Parmi ces méthodes, la classification de textes joue un rôle central dans de nombreuses applications telles que la recherche d'information, la veille scientifique, l'analyse documentaire et l'aide à la décision.

Contrairement à la classification de texte dite *plate*, la classification hiérarchique de textes (CHT) tient compte des relations parent–enfant entre catégories organisées sous forme d'arbre, permettant ainsi une structuration plus fine et plus cohérente de l'information [27]. Par exemple, un document classé dans la catégorie « Sciences » peut être ensuite spécialisé en « Informatique » ou « Biologie ». Ignorer cette structure hiérarchique peut conduire à des incohérences sémantiques et à une propagation d'erreurs, dans la mesure où une mauvaise prédiction à un niveau supérieur affecte l'ensemble des décisions prises aux niveaux inférieurs [4, 13].

Ces dernières années, les grands modèles de langage (GML), tels que BERT ou LLaMA, ont montré des performances remarquables dans de nombreuses tâches de Traitement Automatique du Langage Naturel grâce à l'utilisation de représentations contextuelles profondes [8, 30]. Toutefois, malgré leur efficacité, ces modèles présentent plusieurs limites notables, notamment un coût computationnel élevé, une faible interprétabilité et une forte dépendance à de grands volumes de données annotées [2].

Dans ce contexte, les méthodes à noyau constituent une alternative pertinente et robuste. Ces méthodes reposent sur des fonctions mathématiques permettant de projeter les données dans des espaces de dimension élevée, rendant possible la séparation de données non linéairement séparables à l'aide de classifieurs tels que les machines à vecteurs de support (SVM) [7, 24]. Les noyaux peuvent être linéaires, non linéaires (noyaux polynomial ou gaussien *RBF*) ou encore exploiter directement la structure des données textuelles, comme les *string kernels* et les *tree kernels* [6, 17]. De nombreux travaux ont démontré l'efficacité de ces approches dans divers domaines applicatifs, notamment la classification de documents scientifiques, juridiques et de brevets [12].

L'enjeu de ce mémoire est donc d'analyser dans quelle mesure les différentes familles de noyaux, appliquées à la classification hiérarchique de textes, peuvent constituer une alternative à la fois performante, robuste et explicable aux grands modèles de langage, tout en tenant compte des contraintes structurelles imposées par les hiérarchies de classes [26, 27].

1.2 Objectifs du mémoire

Ce mémoire vise à :

- étudier les fondements théoriques de la classification hiérarchique de textes et des méthodes à noyau ;
- implémenter et comparer différents noyaux (linéaire, polynomial, RBF, *string kernels*, *tree kernels*) dans un cadre de classification hiérarchique ;
- évaluer leurs performances à l'aide de métriques adaptées telles que la *hierarchical precision*, la *hierarchical recall*, le *hierarchical F1-score* et des métriques basées sur le Lowest Common Ancestor (LCA) sur des jeux de données de référence comme RCV1-v2, Web of Science (WOS) et WIPO-alpha ;
- identifier les avantages et les limites de chaque famille de noyaux et proposer des perspectives d'amélioration, notamment leur intégration avec des approches neuronales récentes.

1.3 Structure du mémoire

Le mémoire est organisé comme suit :

- Chapitre 2 : présentation des fondements théoriques du Traitement Automatique du Langage Naturel, incluant les méthodes de représentation des textes et les principales approches de classification ;
- Chapitre 3 : état de l'art sur la classification hiérarchique de textes, les méthodes à noyau et les approches basées sur l'apprentissage profond ;
- Chapitre 4 : description de la méthodologie adoptée, des noyaux étudiés et des métriques d'évaluation utilisées ;
- Chapitre 5 : présentation des résultats expérimentaux, analyse comparative des différentes méthodes et discussion critique ;
- Chapitre 6 : conclusion générale, limites du travail et perspectives de recherche futures.

2. Fondements théoriques du Traitement Automatique du Langage Naturel (TALN)

2.1 Introduction au Traitement Automatique du Langage Naturel(TALN)

2.1.1 Définition et importance du TALN

Le Traitement Automatique du Langage Naturel (TALN), ou Natural Language Processing (NLP) en anglais, est une branche de l'intelligence artificielle (IA) qui permet aux ordinateurs de comprendre, générer et manipuler le langage humain, qu'il s'agisse de texte ou de parole, dans toutes les langues humaines [21]. Cette discipline s'appuie sur la linguistique computationnelle, qui modélise les structures grammaticales [11], combinée à des techniques de machine learning et d'apprentissage profond pour traiter des données non structurées issues de sources variées (e-mails, réseaux sociaux, documents professionnels) [1]. Parmi ses objectifs, on trouve l'extraction d'informations, l'analyse de sentiments, la génération de texte et la classification, y compris dans des contextes hiérarchiques où les catégories sont organisées en structures complexes, un enjeu clé pour des domaines comme le juridique ou la santé [21].

L'importance du TALN découle de l'explosion des données textuelles et vocales générées quotidiennement, nécessitant des outils capables de surmonter les défis des dialectes, argots et irrégularités grammaticales [1]. Il joue un rôle essentiel dans l'automatisation des tâches (ex. classification de documents volumineux) [11], l'optimisation des recherches, et l'engagement client via des chatbots ou assistants virtuels (comme Siri ou Alexa) [21]. Dans les entreprises, le TALN facilite l'analyse de grandes collections de données, soutient la prise de décision dans des secteurs comme la finance ou les soins de santé [1], et répond aux besoins croissants de gestion d'informations structurées hiérarchiquement, renforçant ainsi son impact scientifique et pratique [11].

2.1.2 Évolution et approches principales

2.1.2.1 Approches symboliques et basées sur les règles

Les premières tentatives en traitement automatique du langage naturel (TALN) reposaient sur des approches symboliques et basées sur des règles (rule-based). Ces systèmes utilisaient des ensembles de règles grammaticales et des dictionnaires, comme des règles pour identifier les noms propres, pour analyser et manipuler le langage. L'exemple le plus célèbre est le chatbot ELIZA [32], qui simulait une conversation en reformulant les phrases de l'utilisateur. Malgré son succès initial, cette méthode présentait des limites majeures : elle ne pouvait gérer ni l'ambiguïté

ni les subtilités du langage. De plus, chaque nouvelle règle devait être codée manuellement, ce qui rendait le système rigide et difficilement extensible. Malgré ces contraintes, ces systèmes rudimentaires ont introduit des arbres syntaxiques, un concept précurseur des méthodes à noyaux pour les hiérarchies.

2.1.2.2 Approches statistiques et machine learning classique

Pour dépasser ces limitations, le TALN a évolué vers des approches statistiques dans les années 1980. L'idée était d'exploiter de vastes corpus de données pour apprendre les probabilités d'occurrence et les modèles linguistiques. Les techniques de machine learning classique, telles que la classification naïve bayésienne(Naive Bayes), les chaînes de Markov cachées (HMM) pour modéliser les séquences temporelles, ou les machines à vecteurs de support (SVM), ont permis de créer des modèles plus robustes et capables de généraliser. IBM Watson illustre cette période : capable de traiter et comprendre des milliards de documents, il pouvait répondre avec précision à des questions en langage naturel [9]. Les SVM, notamment avec des noyaux adaptés, ont permis de modéliser des relations complexes, facilitant ainsi la classification hiérarchique.

2.1.2.3 Approches basées sur le deep learning et les modèles pré-entraînés

La révolution la plus récente est venue du deep learning, qui exploite des réseaux de neurones profonds pour dépasser les limitations des approches statistiques. Les modèles récurrents (RNN) et, plus récemment, les architectures Transformer ont permis une compréhension sémantique plus fine et une génération de texte performante. Les modèles pré-entraînés sur des jeux de données massifs, tels que BERT pour la compréhension [8] et GPT pour la génération [20], capturent désormais le contexte d'un mot dans une phrase entière, offrant des performances remarquables sur des tâches complexes. L'apprentissage par transfert permet également de spécialiser ces modèles pré-entraînés sur des tâches spécifiques avec relativement peu de données. Cependant, leur complexité limite l'explicabilité ; par exemple, dans la classification de documents juridiques, leur manque d'explicabilité contraste avec les noyaux.

Ainsi, l'évolution du TALN vers des modèles plus adaptatifs ouvre la voie à des applications sophistiquées comme la classification hiérarchique de textes, où les méthodes à noyaux jouent un rôle croissant.

2.2 Classification de textes

La classification de textes (CT) est une tâche centrale du Traitement Automatique du Langage Naturel (TALN), décrite par [33] comme un processus supervisé visant à associer des documents textuels à un ensemble prédéfini d'étiquettes. Cette tâche, largement étudiée dans la communauté NLP, sous-tend de nombreuses applications pratiques telles que l'étiquetage de sujets ou l'analyse de sentiments [33].

2.2.1 Définition et typologie

Selon [33], la CT repose sur des algorithmes d'apprentissage supervisé qui mappent des textes à des étiquettes. Trois typologies principales émergent :

- Classification binaire : Assignation à l'une de deux catégories mutuellement exclusives (ex. spam/non-spam). C'est le cas le plus simple, où un document est classé dans l'une ou l'autre des deux options [33].
- Classification multiclasse : Attribution d'une seule catégorie parmi plusieurs options mutuellement exclusives (ex. sport, politique) [33].
- Classification multi-étiquettes : Possibilité d'associer plusieurs catégories à un document (ex. action et comédie), un cas complexe où les étiquettes peuvent être interdépendantes, comme dans la classification hiérarchique (HTC) [33].

Ces typologies posent les bases pour explorer les structures hiérarchiques des étiquettes.

2.2.2 Représentations textuelles

[33] soulignent que la représentation textuelle est un concept clé influençant les performances des classificateurs. Les principales approches incluent :

- BoW (Bag of Words) : Une méthode qui transforme les textes en vecteurs de fréquences, ignorant l'ordre des mots [33].
- TF-IDF (Term Frequency-Inverse Document Frequency) : Une technique qui pondère les termes selon leur pertinence dans le corpus [33].
- Embeddings : Vecteurs denses capturant des relations sémantiques, divisés en :
  - Non contextuels : Comme Word2Vec [19] et GloVe [22], avec un vecteur fixe par mot.
  - Contextuels : Comme BERT [8] et LLaMA [29], adaptés au contexte, comme exploré dans les stratégies d'embedding récentes.

Ces représentations jouent un rôle crucial dans les performances des tâches de classification, y compris la HTC.

3. État de l'art sur la Classification Hiérarchique de Textes (CHT)

Ce chapitre présente une revue exhaustive des travaux existants dans le domaine de la Classification Hiérarchique de Textes (CHT). Nous aborderons les concepts fondamentaux, les stratégies de modélisation, les méthodes à noyaux comme solution aux défis spécifiques, leurs applications, et leur comparaison avec les approches neuronales, dans le but d'évaluer leur pertinence pour la CHT.

3.1 Classification hiérarchique de textes (CHT)

La Classification Hiérarchique de Textes (CHT) est une extension complexe de la classification de textes standard, où les catégories ne sont pas indépendantes mais organisées selon une structure hiérarchique (arborescence ou graphe acyclique dirigé - DAG(Graphe Acyclique Dirigé)). Cette tâche reflète mieux la complexité inhérente de nombreux domaines réels, tels que l'organisation de documents médicaux (MeSH), les systèmes de classification de brevets (IPC, WIPO-alpha), ou la structuration de bibliothèques numériques. La CHT vise à exploiter cette structure de classes pour améliorer la précision et la cohérence des prédictions, garantissant qu'un document classé dans une sous-catégorie appartienne également à ses catégories parentes [33]. Elle se distingue des approches de classification plate en considérant explicitement les relations parent-enfant, ce qui pose des défis algorithmiques spécifiques mais offre des avantages significatifs en termes de précision et d'interprétabilité. Les fondements de la CHT ont été explorés par des travaux pionniers tels que ceux de Koller et Sahami [13], et sont synthétisés dans des revues complètes [27, 33].

3.1.1 Modèles locaux et globaux

Les méthodes de CHT peuvent être globalement classées en deux grandes catégories principales, basées sur la manière dont elles abordent la structure hiérarchique des classes et la complexité des dépendances entre elles [27, 33]. La distinction entre ces approches a été analysée en profondeur dans la littérature [5].

3.1.1.1 Modèles locaux

Les approches locales décomposent le problème global de CHT en une série de problèmes de classification plus petits et indépendants, chacun étant résolu par un classifieur dédié [27, 33].

- Local Classifier per Node (LCN) : Un classifieur est entraîné pour chaque nœud (catégorie) de la hiérarchie. Pour un document donné, chaque classifieur décide indépendamment si le document appartient ou non à sa catégorie respective. Cette approche est simple à mettre en œuvre et permet de gérer des hiérarchies de grande taille, mais elle ignore les

relations structurelles entre les nœuds, ce qui peut entraîner des prédictions incohérentes (par exemple, un document classé dans une catégorie enfant mais pas dans son parent direct) [33].

- Local Classifier per Parent Node (LCPN) : Un classifieur est entraîné pour chaque nœud parent afin de distinguer ses enfants directs. Par exemple, si une catégorie "Véhicules" a des enfants "Voitures" et "Motos", un classifieur est entraîné pour différencier ces deux sous-catégories pour les documents déjà étiquetés comme "Véhicules". Cette méthode renforce la cohérence parent-enfant par construction mais peut propager les erreurs des niveaux supérieurs vers les niveaux inférieurs [33].
- Local Classifier per Level (LCL) : Un ensemble de classifieurs est entraîné pour chaque niveau de la hiérarchie. Par exemple, un classifieur pour le niveau 1 (catégories racines), un autre pour le niveau 2 (leurs enfants), etc. Cette approche tente d'améliorer la cohérence à travers les niveaux, mais elle peut également souffrir de la propagation des erreurs [27].

Les modèles locaux sont souvent plus rapides à entraîner et plus flexibles face aux modifications de la hiérarchie, mais leur principale faiblesse réside dans leur incapacité à capturer les dépendances globales de la structure, conduisant potentiellement à des violations de cohérence.

3.1.1.2 Modèles globaux

Contrairement aux approches locales, les modèles globaux entraînent un classifieur unique qui prend en compte l'intégralité de la structure hiérarchique et les dépendances entre les classes dès l'étape d'apprentissage [27, 33].

- Flat Classification (FC) : Bien que "plate", cette approche peut être considérée comme un cas limite de modèle global. Elle ignore complètement la hiérarchie en transformant le problème de CHT en un problème de classification multi-étiquettes standard (si les catégories feuilles sont transformées en étiquettes indépendantes). Elle est simple mais perd toute l'information structurelle.
- Global Classifier : Ces modèles intègrent directement la structure hiérarchique dans leur fonction objectif ou leur architecture. Par exemple, certains modèles peuvent utiliser des régularisations basées sur la hiérarchie, ou des architectures neuronales qui propagent des informations entre les nœuds parents et enfants. L'avantage majeur est la garantie de cohérence hiérarchique et potentiellement une meilleure précision grâce à l'exploitation de plus d'informations. Cependant, ils sont souvent plus complexes à concevoir, plus coûteux en calcul, et moins flexibles face aux changements de hiérarchie [33]. Un exemple de cette approche est l'utilisation de fonctions de coût qui pénalisent davantage les erreurs sur des niveaux hiérarchiques plus élevés ou les incohérences [31].

Le choix entre approches locales et globales dépend du compromis souhaité entre la complexité du modèle, la cohérence des prédictions et la taille de la hiérarchie. Pour des hiérarchies profondes et complexes, les modèles globaux sont théoriquement plus adaptés, bien que leur implémentation puisse être plus ardue.

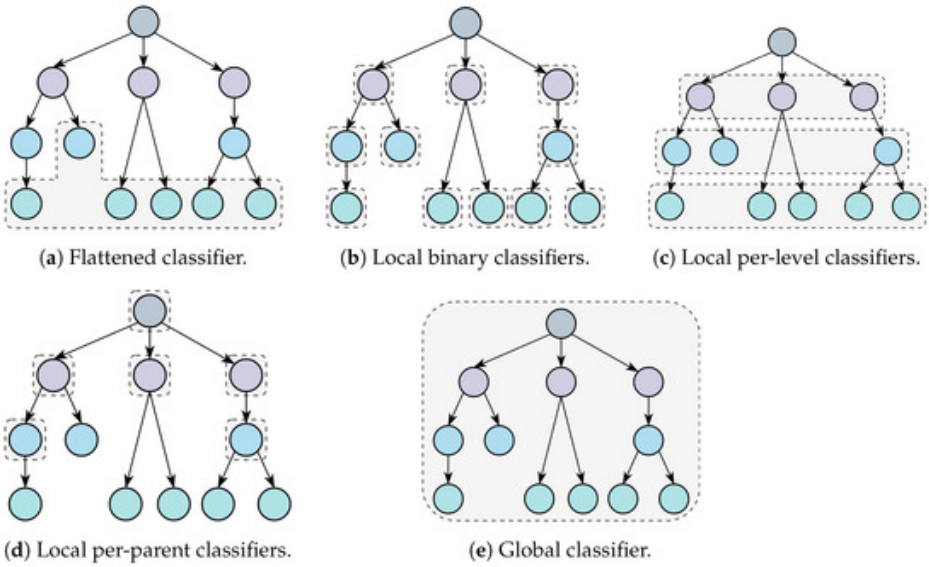

FIGURE 3.1 – Approches courantes en Classification Hiérarchique de Textes (CHT) : comparaison des modèles locaux (LCN, LCPN, LCL) et globaux, ainsi que des principales stratégies d'inférence top-down et bottom-up (adapté de [33]).

Afin de synthétiser les caractéristiques principales des différentes approches de classification hiérarchique de textes, le Tableau 3.1 présente une comparaison entre les modèles locaux et globaux, en mettant en évidence leurs avantages et leurs limites.

TABLE 3.1 – Comparaison des modèles locaux et globaux en classification hiérarchique de textes

Approche	**Description**	**Avantages**	**Limites**
LCN (Local Classifier per Node)	Un classifieur est entraîné pour chaque nœud de la hiérarchie, indépendamment des autres.	Simplicité de mise en œuvre, bonne scalabilité pour les grandes hiérarchies.	Cohérence parent–enfant non garantie, risque élevé d'incohérences hiérarchiques.
LCPN (Local Classifier per Parent Node)	Un classifieur est entraîné pour chaque nœud parent afin de distinguer ses enfants directs.	Cohérence parent–enfant renforcée, bonne interprétabilité des décisions.	Propagation des erreurs depuis les niveaux supérieurs.
LCL (Local Classifier per Level)	Un ensemble de classifieurs est entraîné pour chaque niveau de la hiérarchie.	Cohérence partielle entre les niveaux, réduction du nombre de classifieurs.	Propagation des erreurs entre niveaux, dépendance forte aux prédictions précédentes.
Modèles globaux	Un classifieur unique intègre l'ensemble de la structure hiérarchique lors de l'apprentissage.	Cohérence hiérarchique garantie, exploitation globale des dépendances.	Complexité élevée, coût computationnel important, faible flexibilité.

Tableau inspiré des synthèses proposées dans [5, 27, 33].

3.1.2 Stratégies top-down et bottom-up

Au-delà de la distinction entre modèles locaux et globaux, les méthodes de CHT peuvent également être caractérisées par la stratégie d'inférence employée pour parcourir la hiérarchie et effectuer les prédictions [27, 33]. Ces stratégies définissent l'ordre dans lequel les décisions de classification sont prises à travers les différents niveaux, influençant directement la cohérence des prédictions. Il est important de noter que si les termes "top-down" et "bottom-up" sont souvent utilisés pour décrire le fonctionnement de certains classifieurs hiérarchiques, Silla et Freitas [27] soulignent que ces approches sont principalement des méthodes pour gérer ou corriger les incohérences de prédiction lors de la phase de test, plutôt que des stratégies d'apprentissage complètes en elles-mêmes.

3.1.2.1 Stratégie Top-Down (De haut en bas)

La stratégie top-down est l'une des approches les plus intuitives pour l'inférence en CHT, souvent associée aux classifieurs locaux. Elle commence par la classification d'un document aux niveaux les plus élevés de la hiérarchie, puis progresse séquentiellement vers les niveaux inférieurs [27, 33].

- Processus : Un classifieur de haut niveau (souvent au niveau racine) assigne le document à une ou plusieurs catégories initiales. Par la suite, pour chaque catégorie prédite, un classifieur spécifique au nœud ou au parent (comme un Local Classifier Per Parent Node - LCPN) est activé pour prédire les sous-catégories pertinentes au sein de cette branche. Ce chemin de décision est poursuivi niveau par niveau jusqu'à l'atteinte des nœuds feuilles ou un critère d'arrêt défini. Cette approche impose une cohérence "parent-enfant" où une prédiction à un niveau inférieur implique nécessairement l'appartenance à toutes les catégories parentes sur le chemin [33].
- Avantages : L'avantage principal de la stratégie top-down est qu'elle garantit naturellement la cohérence hiérarchique des prédictions. Elle est également plus facile à comprendre et à interpréter, car elle suit un flux de décision logique du général au spécifique [33].
- Inconvénients : La principale faiblesse de la stratégie top-down est la propagation des erreurs [27]. Une erreur de classification commise à un niveau supérieur de la hiérarchie ne peut pas être corrigée aux niveaux inférieurs, même si les classifieurs des niveaux inférieurs sont intrinsèquement précis. Cela peut sévèrement limiter la performance globale, surtout dans les hiérarchies profondes où les chemins sont longs.

3.1.2.2 Stratégie Bottom-Up (De bas en haut)

La stratégie bottom-up inverse le processus de classification, en commençant par des prédictions aux niveaux les plus granulaires (souvent les nœuds feuilles ou les catégories les plus spécifiques) et en remontant progressivement vers les catégories parentes jusqu'à la racine [33].

- Processus : Initialement, des classifieurs prédisent les catégories les plus spécifiques (feuilles ou niveaux proches des feuilles) auxquelles le document pourrait appartenir. Les informations de ces prédictions granulaires sont ensuite agrégées ou propagées vers le haut. Les classifieurs des niveaux supérieurs utilisent ces informations pour décider de l'appartenance aux catégories parentes

successives. Ce processus ascendant peut impliquer des mécanismes complexes pour consolider les évidences des enfants vers leurs parents.

- Avantages : Cette approche est généralement moins sensible à la propagation des erreurs des niveaux supérieurs, car les décisions initiales se concentrent sur des caractéristiques plus fines. Elle a le potentiel de capter des détails spécifiques au niveau des feuilles qui seraient dilués ou ignorés par des classifieurs de haut niveau [27].
- Inconvénients : La difficulté majeure de la stratégie bottom-up est d'assurer la cohérence hiérarchique des prédictions ascendantes. Il est plus complexe de garantir qu'une prédiction faite à un niveau inférieur soit systématiquement validée et propagée à tous ses ancêtres sans introduire d'incohérences. De plus, elle peut être plus vulnérable au déséquilibre des classes au niveau des feuilles (où de nombreuses catégories peuvent avoir peu d'exemples) et peut entraîner un coût computationnel élevé en raison de l'agrégation des nombreuses prédictions de bas niveau [33].

3.1.2.3 Stratégies Hybrides

Pour pallier les limitations des approches pures top-down et bottom-up, des stratégies hybrides ont été développées. Ces méthodes cherchent à combiner les avantages de chaque direction. Par exemple, une approche pourrait initier un processus top-down pour réduire l'espace de recherche des catégories pertinentes, suivi d'une phase bottom-up pour affiner les prédictions au niveau le plus détaillé. Inversement, une classification initiale de bas niveau pourrait être réajustée par des contraintes de cohérence de haut niveau. Ces stratégies combinées visent à optimiser la performance globale de la CHT en tirant parti de la complémentarité des informations structurelles.

3.1.2.4 Illustration des stratégies d'inférence

Pour clarifier les différences entre les approches Top-Down et Bottom-Up, la figure ci-dessous illustre la propagation des prédictions à travers la hiérarchie :

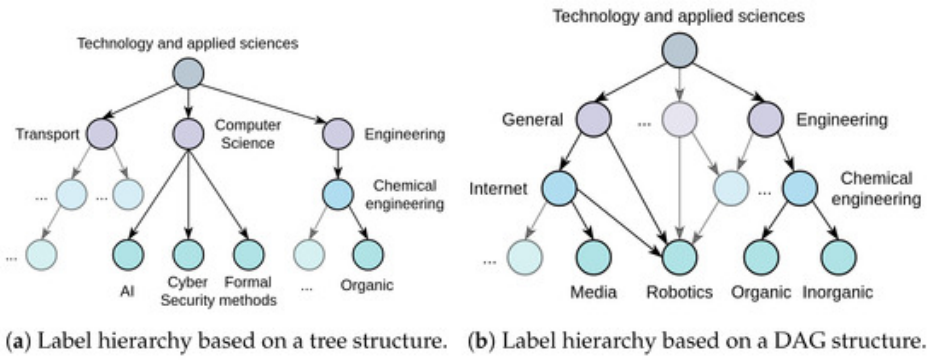

FIGURE 3.2 – Illustration des stratégies d'inférence en CHT : Top-Down et Bottom-Up. Les flèches indiquent la direction de propagation des prédictions à travers la hiérarchie.

3.1.3 Défis spécifiques en CHT

Malgré ses avantages en termes de cohérence et de granularité, la Classification Hiérarchique de Textes (CHT) présente des défis spécifiques qui la distinguent de la classification plate, comme le soulignent [33] et [27]. Ces obstacles, particulièrement pertinents dans des domaines à hiérarchies complexes comme la médecine, le droit, et les brevets exigent des modèles robustes, notamment en intégrant des approches comme les méthodes à noyaux.

3.1.3.1 Propagation des erreurs

La propagation des erreurs est un défi critique, surtout dans les architectures top-down [27]. Une erreur au niveau supérieur (ex. une catégorie parente mal classée) se transmet aux niveaux inférieurs, empêchant l'atteinte de la sous-catégorie correcte, même avec des classifieurs précis. [33] notent que la précision des niveaux profonds dépend des prédictions initiales. Des techniques de correction des chemins de classification ou de révision des prédictions sont explorées pour limiter cet effet.

3.1.3.2 Déséquilibre des classes (Class Imbalance)

Le déséquilibre des classes est amplifié dans les hiérarchies, les catégories supérieures étant souvent surreprésentées tandis que les nœuds feuilles sont rares [27]. [33] soulignent que cela biaise les classifieurs vers les classes majoritaires, réduisant la précision pour les catégories spécifiques (ex. diagnostics médicaux ou brevets détaillés). Des méthodes comme le suréchantillonnage, le sous-échantillonnage, ou des fonctions de perte pondérées sont proposées, bien que leur efficacité varie selon les cas.

3.1.3.3 Complexité computationnelle et gestion des données

La CHT accroît la complexité computationnelle, selon [33] :

- Taille de la hiérarchie : Des structures comme MeSH, avec des milliers de catégories, rendent l'entraînement et l'inférence coûteux, surtout pour les modèles globaux.
- Données d'entraînement : Les annotations hiérarchiques sont coûteuses et rares, limitant la qualité des données, notamment pour les niveaux inférieurs.
- Temps d'inférence : Les approches top-down nécessitent des prédictions séquentielles, ralentissant les classifications en temps réel.

Ces défis appellent des modèles optimisés, où les méthodes à noyaux pourraient jouer un rôle en modélisant efficacement les relations hiérarchiques.

3.2 Méthodes à Noyaux (Kernel Methods) en TALN

Les méthodes à noyaux, ou "Kernel Methods", sont une classe d'algorithmes d'apprentissage automatique puissante, avec les Machines à Vecteurs de Support (SVM) comme représentants majeurs. Elles permettent de traiter des données non-linéairement séparables en projetant implicitement les données

dans un espace de plus haute dimension, offrant robustesse et interprétabilité dans le Traitement du Langage Naturel (TLN). Ces méthodes sont particulièrement pertinentes pour la Classification Hiérarchique de Textes (CHT), où elles modélisent les relations entre nœuds hiérarchiques, comme exploré par [33].

3.2.1 Définition et "Kernel Trick"

Le fondement des méthodes à noyaux repose sur le "Kernel Trick", une technique clé en apprentissage automatique.

3.2.1.1 Définition

Une fonction noyau, notée $K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$, calcule le produit scalaire des vecteurs transformés $\phi(\mathbf{x})$ et $\phi(\mathbf{x}')$ dans un espace de Hilbert de dimension supérieure [23]. Ici, ϕ mappe les vecteurs d'entrée $\mathbf{x}$ et $\mathbf{x}'$ vers un espace de caractéristiques potentiellement infini, permettant aux algorithmes basés sur des produits scalaires d'être "noyautés".

3.2.1.2 L'astuce du noyau (Kernel Trick)

L'avantage du Kernel Trick est de réaliser ces calculs dans l'espace de haute dimension sans le construire explicitement, évitant ainsi une complexité computationnelle excessive. Par exemple, des points 2D non séparables deviennent linéairement séparables en 3D (ex. via la distance à l'origine), sans calculer cette dimension [3, 7]. Ses avantages incluent :

- Modélisation de relations non-linéaires complexes.
- Évitement de la malédiction de la dimension.
- Flexibilité pour intégrer des similarités spécifiques.

Dans le TLN, cette approche est cruciale pour modéliser les structures sémantiques, notamment dans la CHT avec des noyaux comme les string kernels et tree kernels [33].

3.2.2 Noyaux classiques

Après avoir défini le "Kernel Trick" et son rôle, il est essentiel de comprendre les différentes fonctions noyau couramment utilisées. Ces noyaux permettent de calculer la similarité entre les vecteurs de caractéristiques dans un espace de dimension supérieure, sans avoir à explicitement le construire. Le choix du noyau est crucial car il détermine la nature des relations non-linéaires que le modèle peut capturer. Nous allons présenter les noyaux les plus fondamentaux, largement utilisés dans diverses applications d'apprentissage automatique, y compris le Traitement du Langage Naturel (TALN) [23].

3.2.2.1 Noyau Linéaire (Linear Kernel)

Le noyau linéaire est le plus simple des noyaux et correspond au produit scalaire standard des vecteurs d'entrée. Il est défini comme suit :

$$K(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$$

Ce noyau est équivalent à l'application d'un classifieur linéaire (comme un SVM linéaire) directement sur les données d'entrée sans aucune transformation dans un espace de plus haute dimension. Il est efficace lorsque les données sont déjà linéairement séparables ou que la relation entre les caractéristiques est principalement linéaire. Ses avantages incluent sa simplicité, sa rapidité de calcul et son interprétabilité. Cependant, sa capacité à modéliser des relations complexes est limitée [23].

3.2.2.2 Noyau Polynomial (Polynomial Kernel)

Le noyau polynomial permet de modéliser des relations non-linéaires en projetant implicitement les données dans un espace de caractéristiques polynomial. Il est défini par la formule :

$$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$$

où d est le degré du polynôme et c est un terme constant (souvent 0 ou 1). En augmentant le degré d, le noyau polynomial peut capturer des relations de plus en plus complexes et non-linéaires.

- Avantages : Il offre plus de flexibilité que le noyau linéaire pour les données non linéairement séparables. Le degré d est un hyperparamètre qui peut être ajusté pour contrôler la complexité du modèle.
- Inconvénients : Un degré d trop élevé peut entraîner un surapprentissage (overfitting), et le calcul peut devenir coûteux à mesure que d augmente, en particulier avec des données de haute dimension. Le choix de c influence également les performances [23].

3.2.2.3 Noyau Gaussien (Radial Basis Function - RBF Kernel)

Le noyau Gaussien, ou Radial Basis Function (RBF) kernel, est l'un des noyaux les plus populaires et les plus puissants. Il est capable de mapper les données dans un espace de dimension infinie, permettant une grande flexibilité dans la modélisation de relations non-linéaires complexes. Sa formule est donnée par :

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right)$$

où $\|\mathbf{x} - \mathbf{x}'\|^2$ est la distance euclidienne au carré entre les deux vecteurs d'entrée, et γ (gamma) est un paramètre positif qui contrôle l'influence de chaque point d'entraînement. Une petite valeur de γ signifie une grande influence (modèle plus "lisse"), tandis qu'une grande valeur de γ signifie une influence locale (modèle plus "complexe" et potentiellement surajusté).

- Avantages : Le noyau RBF est très polyvalent et peut gérer des données avec des relations complexes non-linéaires. Il fonctionne bien sur une large gamme de problèmes et est souvent le noyau par défaut à essayer [23].
- Inconvénients : Le choix optimal du paramètre γ est crucial et nécessite souvent une validation croisée. Il peut être sensible au surapprentissage si γ est trop élevé, et son interprétabilité est moindre par rapport aux noyaux linéaires. De plus, il peut être coûteux en calcul sur de très grands jeux de données.

Le noyau RBF est souvent considéré comme un bon point de départ car il peut approximer la performance d'autres noyaux dans certaines conditions et est moins susceptible de surapprendre si ses hyperparamètres sont bien réglés.

3.2.3 Noyaux spécifiques au TALN

Les noyaux classiques tels que le linéaire, le polynomial ou le Gaussien (RBF) mesurent la similarité entre des vecteurs de caractéristiques numériques. Cependant, le Traitement du Langage Naturel (TALN) manipule des données intrinsèquement structurées (séquences de mots, phrases, paragraphes, voire des arbres de dépendances syntaxiques ou sémantiques). Pour exploiter pleinement cette richesse structurelle, des noyaux spécifiques ont été développés pour le TALN. Ces noyaux permettent de calculer la similarité directement entre des objets linguistiques complexes, sans passer par une représentation vectorielle explicite qui pourrait perdre des informations structurelles cruciales.

TABLE 3.2 – Comparaison des principaux noyaux utilisés en Traitement du Langage Naturel (inspiré de [27, 33])

Noyau	**Type de données**	**Avantages**	**Limites**
Linéaire	Vecteurs (TF-IDF)	Simple, rapide, interprétable, efficace sur données textuelles de grande dimension	Ne capture pas les relations non linéaires
Polynomial	Vecteurs	Capture des relations non linéaires entre caractéristiques	Sensible au degré, risque de surapprentissage
RBF (Gaussien)	Vecteurs	Très flexible, capable de modéliser des frontières complexes	Coût computationnel élevé, peu interprétable
String Kernel	Séquences de texte	Capture des motifs lexicaux et syntaxiques complexes sans vectorisation explicite	Calcul intensif, peu scalable
Tree Kernel	Arbres syntaxiques ou hiérarchiques	Exploite explicitement la structure syntaxique et hiérarchique des textes	Nécessite une analyse syntaxique préalable, coût élevé

3.2.3.1 Noyaux de Chaînes de Caractères (String Kernels)

Les noyaux de chaînes de caractères (String Kernels) mesurent la similarité entre deux séquences de texte en comptant le nombre de sous-séquences communes qu'elles partagent. Une sous-séquence est une suite de caractères (ou de mots) qui n'est pas nécessairement contiguë. Par exemple, la sous-séquence "ac" est présente dans "abc" et "axc".

- Fonctionnement : Ces noyaux construisent implicitement un espace de caractéristiques où chaque dimension correspond à une sous-séquence possible de caractères (ou de mots) d'une certaine longueur. La valeur de cette dimension pour un texte donné est le nombre d'occurrences de cette sous-séquence dans le texte.

- Exemples courants : Le noyau de sous-séquence (Subsequence Kernel) est l'un des plus connus, mesurant la similarité en fonction des sous-séquences communes avec ou sans "trous" (gaps).
- Avantages : Ils sont capables de capturer des motifs complexes et non contigus dans le texte, sans nécessiter d'ingénierie de caractéristiques manuelle intensive. Ils sont particulièrement utiles pour des tâches comme la classification de textes courts, la détection de spam, ou l'identification de thèmes.
- Inconvénients : Leur complexité computationnelle peut être très élevée pour de longues séquences ou des sous-séquences de grande longueur, nécessitant souvent des algorithmes efficaces pour leur calcul. Ils peuvent aussi être sensibles aux variations de vocabulaire.

3.2.3.2 Noyaux d'Arbres (Tree Kernels)

Les noyaux d'arbres (Tree Kernels) sont conçus pour mesurer la similarité entre des représentations textuelles structurées sous forme d'arbres, telles que les arbres d'analyse syntaxique (parse trees), les arbres de dépendances sémantiques, ou même des hiérarchies de catégories. Ces noyaux évaluent la similarité en comptant le nombre de sous-structures arborescentes communes aux deux arbres.

- Fonctionnement : Un noyau d'arbre projette implicitement les arbres dans un espace de très grande dimension où chaque dimension représente un sous-arbre possible. La valeur est le nombre d'occurrences de ce sous-arbre.
- Exemples courants : Les noyaux d'arbres les plus étudiés incluent le Subset Tree Kernel (STK) qui compte les sous-arbres contigus, et le Partial Tree Kernel (PTK) qui est plus flexible et autorise des sous-arbres non contigus (par exemple, en sautant des nœuds intermédiaires). Ces noyaux sont une forme spécifique de noyaux de graphes, un domaine étudié par des pionniers comme [10] pour la similarité entre structures complexes.
- Avantages : Ils sont extrêmement puissants pour capturer des informations syntaxiques et sémantiques profondes du langage. Ils sont très efficaces pour des tâches comme l'extraction de relations, la classification de phrases basée sur leur structure grammaticale, ou l'analyse de sentiments contextuelle.
- Inconvénients : Ils nécessitent une étape préalable d'analyse syntaxique ou sémantique pour générer la structure arborescente du texte, ce qui peut être coûteux et sujet aux erreurs. Leur complexité computationnelle est généralement plus élevée que celle des noyaux de chaînes, surtout pour de grands arbres.

Les noyaux spécifiques au NLP offrent des outils robustes pour modéliser la complexité intrinsèque du langage, qu'il soit vu comme une séquence ou comme une structure hiérarchique, et sont particulièrement pertinents pour la Classification Hiérarchique de Textes.

3.2.4 Avantages et limitations des méthodes à noyaux

Les méthodes à noyaux, malgré leur puissance, présentent des avantages distincts et des limitations qui influencent leur application en Traitement du Langage Naturel (TLN) et en Classification Hiérarchique de Textes (CHT). Une évaluation approfondie de ces aspects, ancrée dans des travaux comme ceux de [23] et [33], est essentielle pour déterminer leur pertinence face à des approches alternatives.

3.2.4.1 Avantages

Les méthodes à noyaux offrent plusieurs atouts :

- Robustesse et généralisation : Les SVM maximisent la marge, assurant une robustesse au surapprentissage et une bonne généralisation, même avec des données limitées ou bruitées [23].
- Gestion de la non-linéarité : Le Kernel Trick modélise des relations complexes sans transformation explicite, idéal pour les structures sémantiques du texte [33].
- Fondement théorique solide : Leur base mathématique garantit une interprétabilité et une convergence pour certains algorithmes.
- Flexibilité du noyau : Les noyaux structurels (ex. string kernels, tree kernels) s'adaptent aux hiérarchies de la CHT, comme démontré dans des applications médicales [33].

3.2.4.2 Limitations

Cependant, elles comportent des contraintes :

- Coût computationnel élevé : La matrice de noyau (N×N) devient impraticable pour de grands datasets, nécessitant des approximations [23].
- Sélection critique du noyau : Le choix (linéaire, RBF, tree kernels) et le réglage des hyperparamètres exigent une validation croisée, un défi dans la CHT [33].
- Interprétabilité limitée : Les noyaux complexes (ex. RBF) sont moins intuitifs que les modèles linéaires.
- Gestion des multi-classes : Les stratégies "un contre un" ou "un contre tous" compliquent les hiérarchies profondes.

Ces avantages et limitations positionnent les méthodes à noyaux comme des outils puissants pour la CHT, notamment via les noyaux structurels. Toutefois, leur coût et la sélection du noyau restent des défis, ouvrant la voie à des hybrides avec des approches neuronales [33].

3.3 Applications des méthodes à noyaux en Classification Hiérarchique de Textes (CHT)

Les méthodes à noyaux, grâce à leur capacité à modéliser des structures complexes et des similarités sémantiques dans des espaces de haute dimension, sont particulièrement efficaces pour la Classification Hiérarchique de Textes (CHT). Elles exploitent des noyaux spécifiques comme les string kernels et tree kernels, rendant leur application pertinente dans divers domaines, comme détaillé par [33].

3.3.1 Applications en Médecine

Le domaine médical produit d'amples données textuelles (dossiers, articles biomédicaux) organisées via des hiérarchies comme MeSH. Les méthodes à noyaux facilitent l'indexation automatique pour PubMed ou l'aide au diagnostic. [28] ont démontré leur efficacité pour assigner des catégories MeSH, tandis que [33] soulignent leur rôle dans l'amélioration de la précision.

3.3.2 Applications en Droit

Dans le droit, les tree kernels modélisent les structures syntaxiques des jugements et contrats, aidant à identifier des précédents. Par exemple, elles analysent les arbres de dépendance pour détecter des motifs argumentatifs, améliorant la recherche juridique [33].

3.3.3 Applications en Classification de Brevets

Les string kernels sont utilisées pour classifier les brevets dans des systèmes comme WIPO-alpha ou CIP, détectant des sous-séquences de termes techniques non contiguës (ex. "système de propulsion"). Cela optimise l'indexation, comme noté par [33].

Ces applications illustrent la polyvalence des méthodes à noyaux en CHT, bien que leur coût computationnel, comme discuté en section 3.2.4, reste un défi. Elles complètent les approches neuronales, ouvrant des perspectives pour des hybrides efficaces [33].

3.4 Comparaison avec approches neuronales

L'essor des réseaux de neurones profonds et des Transformers a transformé le Traitement du Langage Naturel (TLN), mais les méthodes à noyaux conservent des atouts spécifiques pour la Classification Hiérarchique de Textes (CHT). Une comparaison approfondie, appuyée par [33], éclaire leur pertinence et le potentiel des approches hybrides.

3.4.1 Approches neuronales (Deep Learning et Transformers) : Caractéristiques clés

Les modèles neuronaux, notamment les Transformers, se distinguent par :

- Flexibilité et puissance : Apprentissage de dépendances à longue portée pour la CHT [33].
- Performance : Résultats de pointe sur des tâches de classification.
- Transfert : Fine-tuning efficace de modèles pré-entraînés.
- Scalabilité : Efficace avec de vastes données.

Cependant, leurs limites incluent :

- Coût computationnel élevé.
- Opacité ("boîte noire"), problématique pour la médecine.
- Dépendance aux données massives.
- Sensibilité aux données hors distribution.

3.4.2 Méthodes à Noyaux : Atouts persistants

Les méthodes à noyaux offrent :

- Robustesse : Généralisation avec peu de données, utile pour les nœuds rares en CHT.

- Interprétabilité : Base mathématique solide pour certains noyaux.
- Données réduites : Efficaces avec moins de labels.
- Structures : Exploitation des hiérarchies via tree kernels [33].

3.4.3 Vers des approches hybrides

Les hybrides combinent embeddings de Transformers avec des SVM à noyaux (ex. RBF) pour allier contexte et robustesse. Dans la CHT, les tree kernels pourraient compléter les modèles globaux en capturant les structures hiérarchiques [33]. Afin de synthétiser les différences majeures entre les méthodes à noyaux et les approches neuronales profondes, le Tableau 3.3 présente une comparaison selon plusieurs critères clés fréquemment discutés dans la littérature en Traitement du Langage Naturel [27, 33].

TABLE 3.3 – Comparaison entre les méthodes à noyaux (SVM) et les approches neuronales profondes en Traitement du Langage Naturel

Critère	**Méthodes à noyaux (SVM)**	**Deep Learning / Transformers**
Robustesse aux petits jeux de données	Élevée, bonne généralisation avec peu d'exemples	Faible, nécessite de grandes quantités de données
Interprétabilité	Relativement élevée, fondements mathématiques explicites	Faible, modèles de type « boîte noire »
Gestion de la hiérarchie	Naturellement exploitable via tree kernels ou contraintes hiérarchiques	Partielle, souvent indirecte ou approximative
Scalabilité sur grands corpus	Limitée pour certains noyaux complexes	Très élevée avec des infrastructures adaptées
Capture des dépendances à longue portée	Limitée	Très efficace grâce à l'auto-attention
Coût computationnel	Modéré à élevé selon le noyau	Très élevé (entraînement et inférence)
Besoin en annotations	Faible à modéré	Élevé

Tableau inspiré des synthèses proposées dans [5, 27, 33].

Ce chapitre a démontré que la CHT, avec ses défis comme la propagation des erreurs et le déséquilibre des classes, bénéficie des méthodes à noyaux pour modéliser les hiérarchies via des noyaux structurels (tree kernels, string kernels). Bien que les approches neuronales dominent par leur scalabilité, les hybrides combinant embeddings et noyaux offrent un potentiel prometteur. Les travaux futurs pourraient explorer l'optimisation de ces hybrides pour des hiérarchies complexes, comme abordé dans notre démarche expérimentale.

4. Cadre expérimental et méthodologique de la classification hiérarchique

Ce chapitre présente la méthodologie adoptée pour la classification hiérarchique de textes scientifiques à l'aide de méthodes à noyau appliquées au traitement automatique du langage naturel (TALN). Il décrit successivement le jeu de données, les étapes de prétraitement, la formulation du problème, les modèles utilisés ainsi que les critères d'évaluation.

4.1 Présentation du jeu de données

4.1.1 Source des données

Le jeu de données utilisé dans ce travail est constitué de publications scientifiques issues de la base bibliographique *Web of Science*. Pour chaque document, les champs textuels exploités sont la concaténation du titre et du résumé (*abstract*), ce qui permet de maximiser la densité d'information sémantique fournie aux modèles.

Chaque document est associé à une classification hiérarchique structurée en trois niveaux :

- un niveau parent (*YL1*)
- un niveau intermédiaire (*YL2*)
- un niveau feuille (Y), représentant la spécialité finale.

Les critères d'inclusion ont été strictement définis : seuls les articles disposant d'un chemin hiérarchique complet et valide ($YL1 \rightarrow YL2 \rightarrow Y$) ont été conservés pour garantir la cohérence de l'apprentissage.

Concernant l'écart statistique relevé entre le nombre initial d'articles (11 975) et le nombre réellement utilisé (11 913), celui-ci s'explique par une phase de nettoyage rigoureuse. Cette étape a consisté à supprimer les doublons ainsi que les entrées comportant des textes vides ou inexploitables afin de ne pas biaiser les performances des classifieurs.

Enfin, bien que le déséquilibre des classes soit marqué et inhérent à ce type de corpus scientifique, il a été décidé de traiter les données sans ré-échantillonnage artificiel. Ce choix méthodologique permet de tester la robustesse réelle des SVM et de l'architecture LCPN face à des distributions de données réelles et complexes, reflétant fidèlement la production de la littérature scientifique.

4.1.2 Structure hiérarchique des classes

La hiérarchie des labels suit une structure arborescente dans laquelle chaque document appartient à un chemin unique allant du niveau général (*YL1*) vers le niveau le plus spécifique (Y).

La Figure 4.1 illustre cette organisation hiérarchique, essentielle à la formulation du problème de classification.

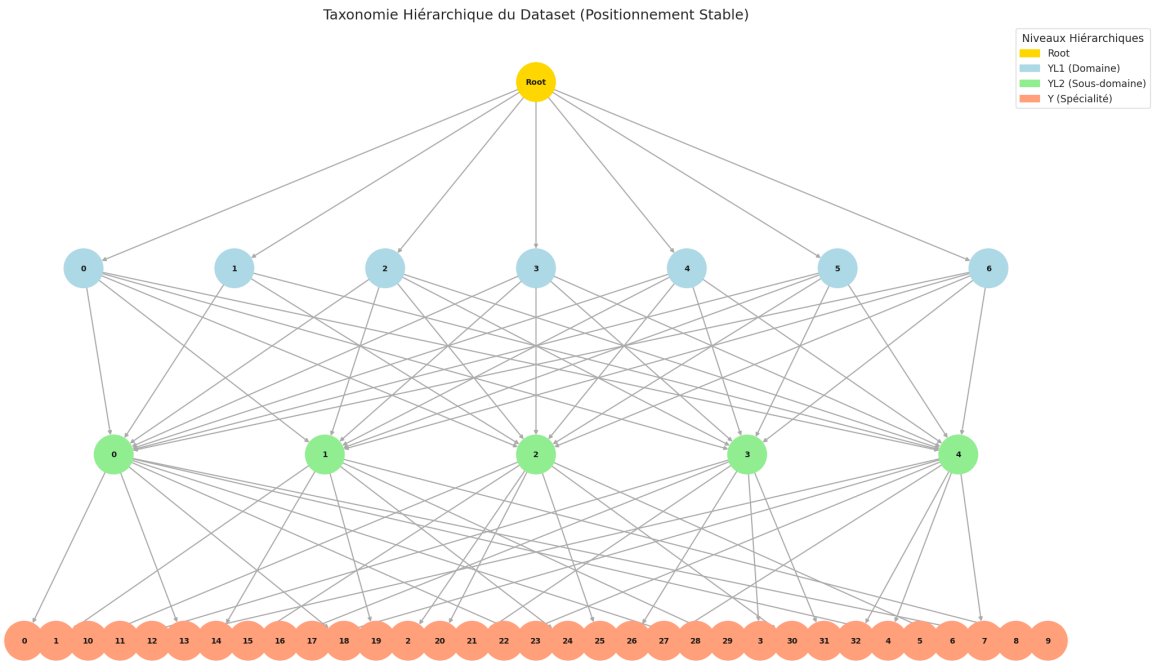

FIGURE 4.1 – Organisation hiérarchique des classes scientifiques selon les niveaux YL1, YL2 et Y (taxonomie LCPN)

4.1.3 Analyse des biais et limites du jeu de données

La constitution du corpus, bien que représentative de la production scientifique indexée jusqu'en 2020, présente des biais intrinsèques qu'il convient de souligner pour garantir la transparence de l'étude et répondre aux exigences de rigueur méthodologique.

- Biais linguistique : Le dataset est exclusivement composé de publications en langue anglaise. Si cela assure une cohérence optimale pour l'application des modèles de plongement de mots (*Word Embeddings*) et de *BERT*, cela limite la généralisation des résultats aux productions scientifiques francophones ou multilingues.
- Biais thématique : Nous observons que la distribution des documents suit la productivité historique des domaines de recherche. Les secteurs de la biomédecine et des technologies de l'information surreprésentent certaines branches de la hiérarchie. Cette densité inégale explique la forte performance sur les classes majoritaires et la difficulté relative des modèles à généraliser sur des niches scientifiques moins documentées.
- Gestion du déséquilibre (Choix du non-rééchantillonnage) : Concernant le déséquilibre des classes, nous avons délibérément choisi de ne pas appliquer de techniques de ré-échantillonnage synthétique (telles que *SMOTE*). Ce choix méthodologique vise à tester la robustesse réelle des classifieurs *SVM* et de l'architecture *BERT* dans des conditions réelles de production documentaire. Dans ce contexte, la rareté d'une classe est considérée comme une information structurelle intrinsèque qu'il est crucial de savoir traiter sans artifice statistique.

4.2 Analyse exploratoire des données

4.2.1 Répartition des classes

Une analyse de la distribution des classes a été réalisée à chaque niveau hiérarchique. Les résultats montrent un déséquilibre marqué, particulièrement au niveau feuille, où certaines classes sont fortement représentées tandis que d'autres restent marginales. Ce déséquilibre constitue un défi majeur pour les modèles de classification supervisée.

Afin de garantir la clarté de l'analyse, les indices numériques utilisés dans les figures suivantes (ex : ID 1 à 7 pour le niveau YL1) correspondent strictement aux domaines thématiques détaillés ultérieurement dans les tableaux de résultats (Tableau 5.1). Cette correspondance permet une lecture croisée entre la volumétrie des classes et les performances prédictives obtenues.

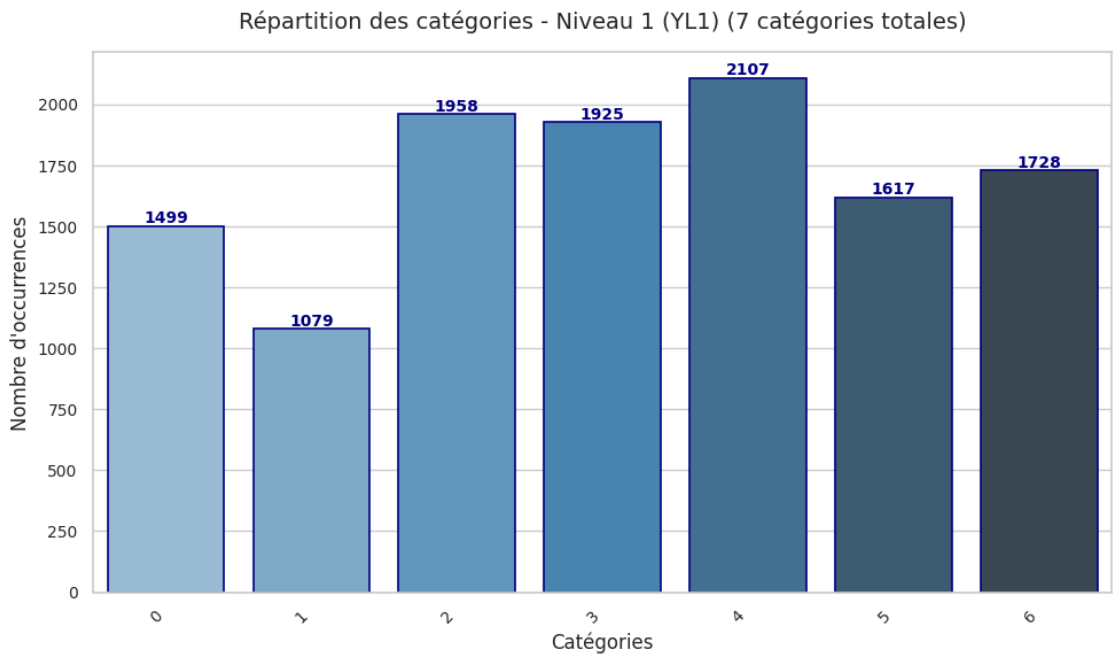

FIGURE 4.2 – Répartition des classes au niveau parent (YL1)

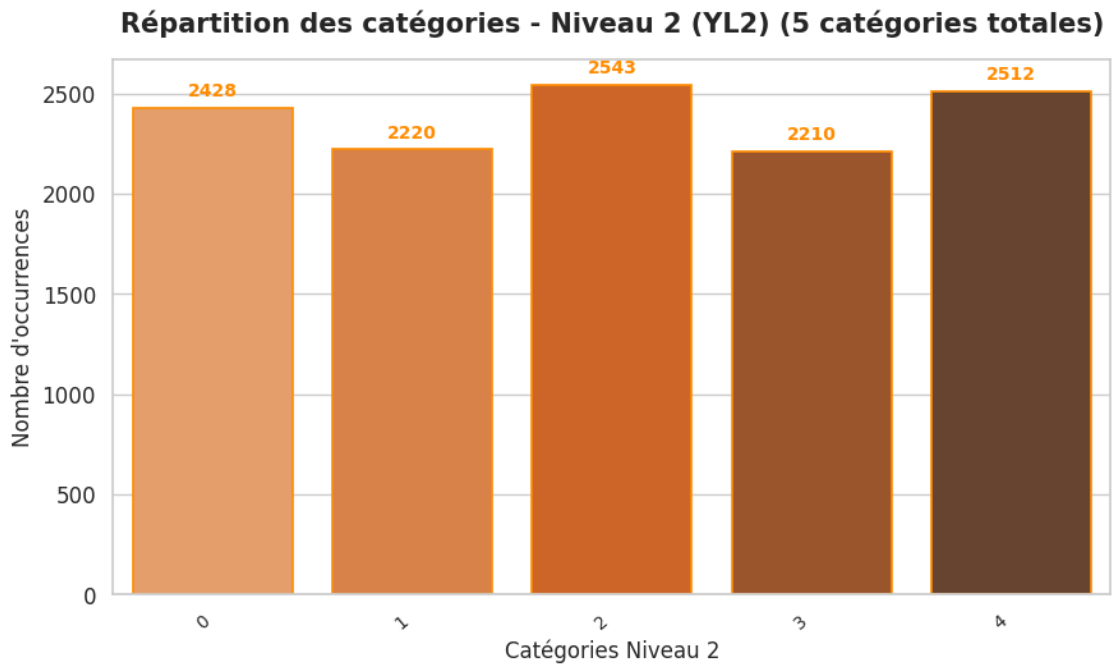

FIGURE 4.3 – Répartition des classes au niveau intermédiaire (YL2)

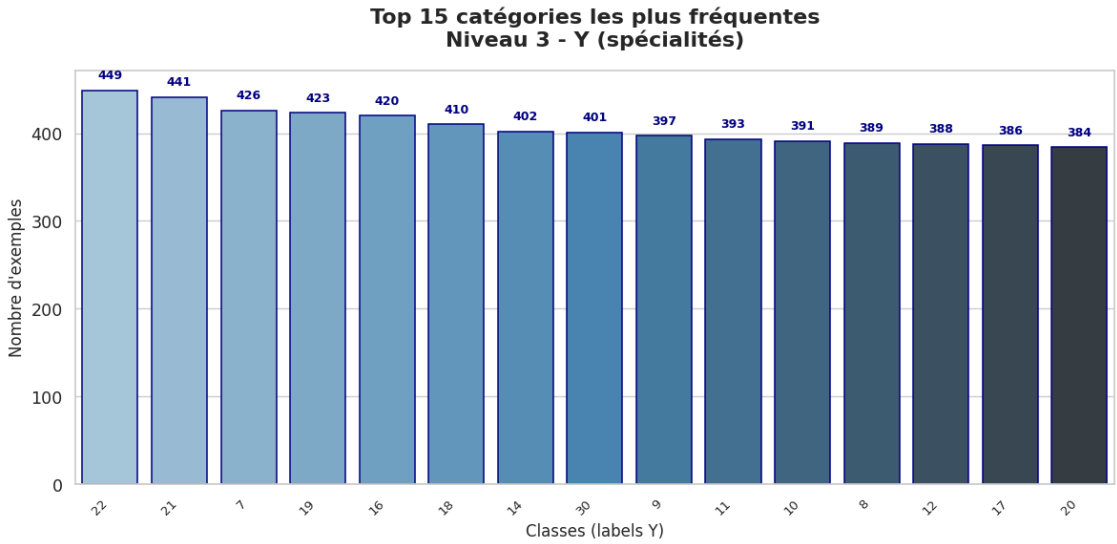

FIGURE 4.4 – Top 10 des classes les plus fréquentes au niveau feuille (Y)

4.2.2 Analyse lexicale

Une analyse lexicale a permis d'identifier les termes les plus fréquents par niveau hiérarchique. Il apparaît que les mots-clés deviennent de plus en plus spécifiques à mesure que l'on descend dans la hiérarchie, confirmant la pertinence d'une approche hiérarchique par rapport à une classification plate.

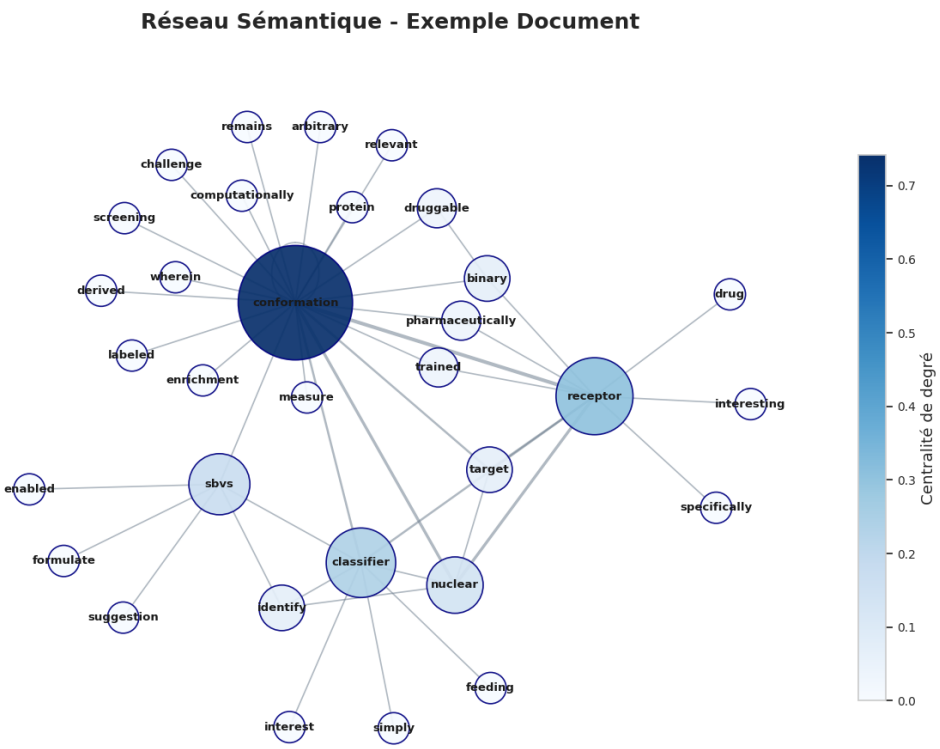

FIGURE 4.5 – Réseau sémantique illustrant la co-occurrence des concepts techniques dans un résumé scientifique

4.2.3 Outil d'exploration dynamique des données

Pour compléter l'analyse lexicale et valider la cohérence sémantique du jeu de données, un outil d'exploration dynamique a été développé. Ce script permet d'extraire de manière aléatoire des documents du corpus et d'afficher leur contenu textuel en regard de leur chemin hiérarchique réel ($YL1 \rightarrow YL2 \rightarrow Y$).

TABLE 4.1 – Exemple de classification hiérarchique pour la classe finale $Y = 6$

Niveau 1 (YL1)	Niveau 2 (YL2)	Classe finale (Y)	Extrait du texte
1	2	6	The paper presents the theoretical background and implementation details of a multi-domain symbolic analysis program developed with the emphasis on linearized hybrid systems combining classical electr...

Les tableaux 4.1 et 4.2 illustrent deux analyses qualitatives complémentaires du modèle de classification hiérarchique. Le premier tableau présente un exemple de document associé à la classe finale $Y = 6$, mettant en évidence la cohérence entre le contenu textuel et le chemin hiérarchique $YL1 \rightarrow YL2 \rightarrow Y$. Le second tableau montre l'exploration d'un document après mélange aléatoire du dataset, où l'index 0 correspond à une branche réelle de la hiérarchie ($YL1 = 6$, $YL2 = 2$, $Y = 30$).

L'utilisation de cet outil d'exploration, fondée sur l'analyse conjointe du contenu textuel et de la structure hiérarchique, confirme la pertinence de l'étiquetage du corpus. En confrontant directement les documents

TABLE 4.2 – Exploration qualitative d'un document après mélange du dataset (index 0)

Index	Niveau parent (YL1)	Niveau intermédiaire (YL2)	Classe feuille (Y)	Extrait du texte
0	6	2	30	This article outlines major developments in knowledge about the human metabolism of ethanol. The results of a large number of controlled experiments aimed at measuring the rate of ethanol elimination from the blood are reported. The factors that influence the rate of ethanol elimination from blood, such as the amount of ethanol ingested, the drinking habits of the subjects, and the effect of food taken together with, or before, drinking were investigated. The slowest rate of ethanol disappearanc...

à leur chemin hiérarchique réel, il est possible de vérifier que les niveaux $YL1$, $YL2$ et Y ne sont pas arbitraires mais correspondent à des thématiques scientifiques cohérentes. Cette étape constitue une validation qualitative essentielle en amont des phases de nettoyage et d'apprentissage automatique, garantissant que la structure hiérarchique du jeu de données est sémantiquement fondée.

4.3 Prétraitement des textes

Le prétraitement des données textuelles dépend du type de représentation vectorielle utilisée. Deux stratégies distinctes ont été adoptées dans ce travail : une adaptée à la représentation statistique TF-IDF et une spécifique aux plongements contextuels BERT.

4.3.1 Prétraitement pour la représentation TF-IDF

Dans le cadre de la vectorisation TF-IDF, les textes ont subi les étapes suivantes :

- conversion en minuscules,
- suppression des caractères spéciaux et des chiffres,
- tokenisation,
- suppression des mots vides (*stopwords*),
- lemmatisation.

Ces opérations permettent de réduire le bruit linguistique, de normaliser le vocabulaire et d'améliorer la qualité des représentations creuses de grande dimension produites par TF-IDF.

4.3.2 Prétraitement pour la représentation BERT

Contrairement aux approches statistiques classiques, les modèles de langage pré-entraînés comme BERT intègrent leur propre mécanisme de tokenisation contextuelle.

Ainsi, aucun nettoyage linguistique avancé (suppression de stopwords ou lemmatisation) n'a été appliqué afin de préserver la structure syntaxique complète des phrases. Les textes ont été uniquement normalisés (suppression des caractères non textuels majeurs), puis traités via le tokenizer officiel du modèle BERT.

Cette différence méthodologique est essentielle, car les représentations contextuelles exploitent les dépendances sémantiques complètes entre les mots.

4.4 Représentation vectorielle

La conversion des textes en vecteurs numériques constitue une étape centrale du processus d'apprentissage automatique. Dans cette étude, deux paradigmes de représentation ont été comparés : une approche statistique creuse (TF-IDF) et une approche contextuelle dense (BERT).

4.4.1 Représentation statistique : TF-IDF

La méthode TF-IDF (Term Frequency – Inverse Document Frequency) pondère chaque terme selon son importance relative dans un document par rapport à l'ensemble du corpus.

Chaque document est ainsi représenté par un vecteur creux de grande dimension, où chaque composante correspond à un terme du vocabulaire.

Cette représentation présente plusieurs avantages :

- simplicité et rapidité de calcul,
- forte interprétabilité,
- efficacité reconnue avec les SVM linéaires pour les tâches de classification textuelle.

Cependant, TF-IDF ne capture pas le contexte sémantique des mots et repose uniquement sur des statistiques de fréquence.

4.4.2 Représentation contextuelle dense : BERT

Afin d'évaluer l'apport des représentations sémantiques profondes, des embeddings issus du modèle pré-entraîné BERT (Bidirectional Encoder Representations from Transformers) ont été utilisés.

BERT est un modèle basé sur l'architecture Transformer, entraîné sur de larges corpus textuels. Il génère des vecteurs denses de dimension fixe (768 dimensions pour la version de base), capturant les relations contextuelles bidirectionnelles entre les mots d'un document.

Dans cette étude :

- les embeddings ont été extraits sans fine-tuning,

- chaque résumé scientifique est représenté par un vecteur dense,
- ces vecteurs sont utilisés comme entrée des classifieurs SVM dans le cadre hiérarchique LCPN.

Contrairement à TF-IDF, les embeddings BERT permettent de modéliser des similarités sémantiques profondes entre documents, indépendamment des variations lexicales.

4.4.3 Comparaison des deux paradigmes

Les deux représentations présentent des caractéristiques structurelles différentes :

- TF-IDF produit des vecteurs creux de très grande dimension.
- BERT produit des vecteurs denses de dimension modérée.

Cette différence structurelle influence directement le comportement des méthodes à noyau, en particulier le noyau RBF, dont la performance dépend de la géométrie de l'espace vectoriel.

4.5 Protocole expérimental comparatif

Afin d'assurer une comparaison équitable entre TF-IDF et BERT, les conditions expérimentales suivantes ont été maintenues constantes :

- même jeu de données (Web of Science),
- même partitionnement (80% entraînement / 20% test),
- même architecture hiérarchique (LCPN),
- mêmes classifieurs (SVM linéaire et SVM RBF),
- mêmes métriques d'évaluation.

La seule variable modifiée entre les expériences est la représentation vectorielle des documents.

Les performances ont été comparées selon :

- l'accuracy à chaque niveau hiérarchique (YL1, YL2, Y),
- un score hiérarchique global sans-faute,
- le temps d'entraînement,
- la stabilité mesurée par validation croisée.

Cette approche expérimentale permet d'isoler l'impact réel du type de représentation sur la performance des modèles hiérarchiques.

4.6 Impact des représentations sur les méthodes à noyau

Le comportement des SVM dépend fortement de la nature des vecteurs d'entrée.

Dans le cas des représentations TF-IDF, le noyau linéaire s'est révélé particulièrement adapté en raison de la forte dimensionnalité et de la nature creuse des données. En revanche, le noyau RBF peut devenir inefficace dans un espace de très grande dimension creuse.

À l'inverse, avec les vecteurs denses issus de BERT, le noyau RBF devient fonctionnel et capable de modéliser des frontières de décision non linéaires plus complexes.

Cette observation met en évidence l'interaction fondamentale entre la géométrie des données et le choix du noyau.

4.7 Ingénierie des labels et partitionnement

Avant l'entraînement des modèles hiérarchiques, chaque document est associé à ses niveaux $YL1$, $YL2$ et Y, garantissant que la hiérarchie parent-enfant soit respectée. Chaque document est ainsi placé dans un chemin unique allant du niveau général (*YL1*) au niveau le plus spécifique (*Y*).

Cette organisation permet aux classifieurs SVM de travailler uniquement sur les documents pertinents pour leur sous-arbre. Elle constitue une étape indispensable pour assurer la cohérence hiérarchique et optimiser l'apprentissage.

Le corpus a ensuite été divisé en ensembles d'entraînement (80%) et de test (20%) pour évaluer les performances de manière impartiale. Le Tableau 4.3 résume le nombre de classes à chaque niveau hiérarchique ainsi que la répartition train/test.

TABLE 4.3 – Résumé des niveaux hiérarchiques et partitionnement du jeu de données

Niveau	Nombre de classes uniques
YL1	7
YL2	5
Y	33

Partitionnement	Nombre d'exemples
Train	9530
Test	2383

4.7.1 Classification plate et hiérarchique

Deux approches de classification ont été considérées :

1. Classification plate Dans la classification plate, chaque document d_i est représenté par un vecteur de caractéristiques $\mathbf{x}_i \in \mathbb{R}^n$, et l'objectif est de prédire directement la classe finale $y_i \in \{1, \ldots, C\}$ à l'aide d'un classifieur $f : \mathbb{R}^n \to \{1, \ldots, C\}$ [27].

La fonction de prédiction d'un classifieur supervisé peut s'écrire :

$$\hat{y}_i = \arg \max_{c \in \{1,\ldots,C\}} P(y_i = c \mid \mathbf{x}_i)$$

où $P(y_i = c \mid \mathbf{x}_i)$ est la probabilité estimée que le document d_i appartienne à la classe c.

2. Classification hiérarchique (LCPN) Dans l'approche hiérarchique de type *Local Classifier Per Node (LCPN)* [27], un classifieur $f_p^{(l)}$ est entraîné pour chaque nœud parent p au niveau l de la hiérarchie. Chaque classifieur prédit les classes enfants $y^{(l+1)}$ à partir des documents appartenant au parent p.

Formellement, si d_i appartient à un parent p au niveau l, la prédiction est donnée par :

$$\hat{y}_i^{(l+1)} = \arg \max_{c \in \mathsf{Enfants}(p)} P(y_i^{(l+1)} = c \mid \mathbf{x}_i, p)$$

Le chemin hiérarchique complet d'un document s'obtient en combinant les prédictions à chaque niveau :

$$\hat{\mathbf{y}}_i = (\hat{y}_i^{(1)}, \hat{y}_i^{(2)}, \ldots, \hat{y}_i^{(L)})$$

où L est la profondeur maximale de la hiérarchie.

Ainsi, la classification hiérarchique exploite explicitement la structure parent-enfant et réduit les incohérences de prédiction par rapport à une approche plate.

4.7.2 Modèles de classification

Les méthodes de classification utilisées dans ce travail reposent principalement sur les machines à vecteurs de support (Support Vector Machines, SVM), qui constituent une approche robuste et largement utilisée pour la classification de données textuelles de grande dimension.

4.7.2.1 Machines à vecteurs de support

Soit un ensemble d'apprentissage $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, où $\mathbf{x}_i \in \mathbb{R}^n$ représente le vecteur de caractéristiques d'un document et $y_i \in \{-1, +1\}$ son étiquette de classe. Le principe fondamental des SVM consiste à déterminer un hyperplan de séparation maximisant la marge entre les classes, défini par :

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

L'optimisation associée au cas linéaire s'écrit :

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i$$

sous les contraintes :

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

où C est un paramètre de régularisation contrôlant le compromis entre la maximisation de la marge et l'erreur de classification [7].

4.7.2.2 Méthodes à noyau

Dans de nombreux cas, les données ne sont pas linéairement séparables dans l'espace original. Les SVM utilisent alors une fonction noyau $K(\mathbf{x}_i, \mathbf{x}_j)$ permettant de calculer implicitement un produit scalaire dans un espace de dimension plus élevée [24].

La fonction de décision s'exprime sous la forme :

$$f(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

où α_i sont les multiplicateurs de Lagrange issus du problème dual.

Dans ce travail, deux noyaux ont été étudiés :

- Noyau linéaire :
$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$$
particulièrement adapté aux données textuelles représentées par des vecteurs TF-IDF de grande dimension [12].
- Noyau RBF (Radial Basis Function) :
$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$
permettant de modéliser des frontières de décision non linéaires complexes.

4.7.2.3 Stratégies de classification hiérarchique

Les SVM ont été intégrés dans un cadre de classification hiérarchique selon l'approche *Local Classifier Per Parent Node (LCPN)*. À chaque nœud parent p de la hiérarchie, un classifieur SVM est entraîné pour discriminer uniquement entre ses classes enfants. La prédiction à un niveau $l+1$ est donnée par :

$$\hat{y}_i^{(l+1)} = \arg \max_{c \in \text{Enfants}(p)} f_p^{(l)}(\mathbf{x}_i)$$

Le chemin hiérarchique final d'un document est obtenu par la composition successive des décisions locales le long de la hiérarchie [27]. Cette stratégie permet de réduire la complexité du problème et de limiter les incohérences de prédiction entre niveaux.

4.7.3 Évaluation des performances

4.7.3.1 Métriques utilisées

Les performances ont été mesurées avec des métriques classiques : précision, rappel et F1-score. Une métrique hiérarchique globale a également été définie pour évaluer la capacité du modèle à prédire correctement l'ensemble du chemin hiérarchique ($YL1 \rightarrow YL2 \rightarrow Y$).

4.7.3.2 Validation croisée

Une validation croisée à k plis a été effectuée pour évaluer la robustesse et la stabilité des modèles, limitant le biais lié à une seule séparation entraînement/test.

5. Résultats et interprétation

Ce chapitre présente et analyse les résultats obtenus à partir des expérimentations menées sur le jeu de données *Web of Science*. L'évaluation porte sur la capacité des modèles à maintenir des performances stables et cohérentes à travers les différents niveaux de la hiérarchie de classes.

Les résultats sont analysés à l'aide de mesures quantitatives et de représentations graphiques, permettant d'évaluer l'efficacité de l'approche de classification hiérarchique basée sur les machines à vecteurs de support à noyau. Une attention particulière est portée à la comparaison des performances entre les différents niveaux hiérarchiques, afin d'analyser l'impact de la structure hiérarchique sur la qualité des prédictions.

5.1 Performances du modèle Baseline (Random Forest)

Avant l'introduction des méthodes à noyau, nous avons évalué une approche de référence (Baseline) basée sur l'algorithme *Random Forest* combiné à une vectorisation TF-IDF. Cette étape permet d'établir un point de comparaison pour mesurer l'apport réel des SVM et de BERT.

5.1.1 Analyse globale par niveau (Évaluation locale)

L'examen des scores de la Baseline met en évidence une performance locale honorable. Le modèle atteint une précision de 90,10 % au niveau YL1, 81,24 % au niveau YL2 et 81,70 % au niveau final Y. Toutefois, il est primordial de préciser que ces scores de 81,70 % sont calculés indépendamment pour chaque niveau. Ils ne reflètent pas encore la capacité du modèle à valider un chemin complet sans erreur. Comme nous le verrons dans l'analyse de la validation croisée (Section 5.6), cette Baseline souffre d'un effet de cascade : dès qu'une erreur survient au niveau parent, la prédiction finale au niveau feuille est invalidée, faisant chuter la précision réelle du chemin complet à seulement 2,31 %.

5.1.2 Analyse granulaire au niveau parent (YL1)

Le graphique 5.1 montre que le modèle atteint une excellente performance au niveau YL1, avec des scores homogènes et élevés pour toutes les classes. Cette homogénéité traduit la facilité à distinguer les grands domaines scientifiques (ex : Physique vs Biologie), même pour un modèle de référence.

Le tableau 5.1 confirme que le graphique reflète fidèlement les performances. La robustesse du modèle au niveau L1 s'explique par la distinction lexicale nette entre les grands domaines , ce qui facilite la classification et assure une base solide avant d'aborder le niveau intermédiaire $YL2$.

5.1.3 Analyse au niveau intermédiaire (YL2)

Au niveau intermédiaire (YL2), la précision globale s'établit à 81,24 %, marquant une légère diminution par rapport au niveau parent YL1 (90,10 %). Cette baisse s'explique par la proximité sémantique accrue entre certains sous-domaines, ce qui rend la séparation des classes plus délicate.

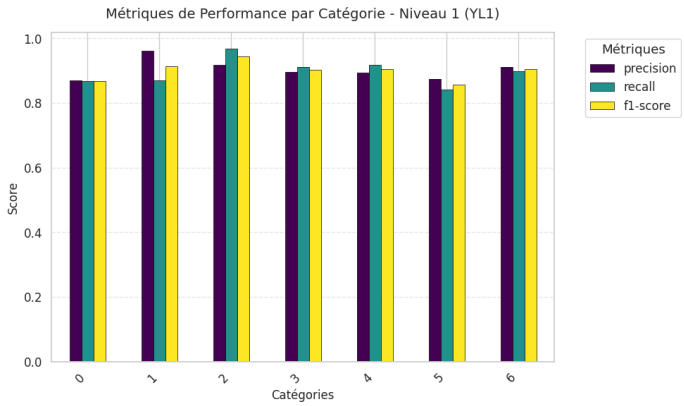

FIGURE 5.1 – Visualisation graphique des métriques au niveau parent YL1 : précision, rappel et F1-score par classe.

TABLE 5.1 – Rapport de performance par classe pour le niveau parent $YL1$

Classe (YL1)	Précision	Rappel	F1-score	Support
0	0.87	0.87	0.87	315
1	0.96	0.87	0.91	230
2	0.92	0.97	0.94	383
3	0.89	0.91	0.90	383
4	0.89	0.92	0.91	423
5	0.87	0.84	0.86	296
6	0.91	0.90	0.90	353
Accuracy	0.901 (2383 exemples)			
Macro avg	0.90	0.90	0.90	2383
Weighted avg	0.90	0.90	0.90	2383

Précision globale (Accuracy) : 90.10%

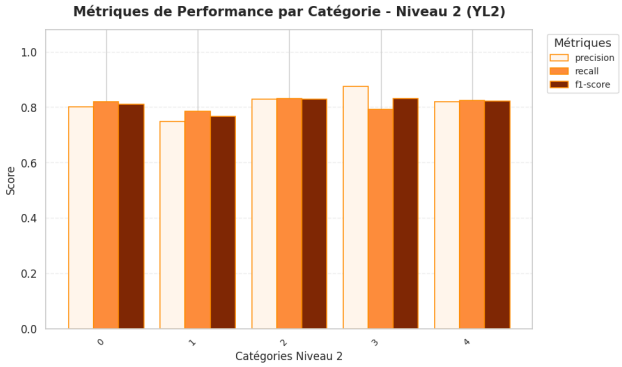

FIGURE 5.2 – Scores par classe pour le niveau intermédiaire YL2 : précision, rappel et F1-score. Les variations traduisent la complexité accrue de classification entre sous-domaines proches.

Les sous-domaines présentant un vocabulaire technique similaire entraînent une légère érosion des scores. Cependant, le modèle conserve des performances homogènes et reste fiable pour la majorité des classes.

TABLE 5.2 – Rapport détaillé par classe pour le niveau intermédiaire $YL2$

Classe (YL2)	Précision	Rappel	F1-score	Support
0	0.80	0.82	0.81	507
1	0.75	0.79	0.77	415
2	0.83	0.83	0.83	545
3	0.87	0.79	0.83	402
4	0.82	0.82	0.82	514
Accuracy	0.812 (2383 exemples)			
Macro avg	0.81	0.81	0.81	2383
Weighted avg	0.81	0.81	0.81	2383

Précision globale (Accuracy) : 81.24%

En résumé, la transition de YL1 à YL2 illustre comment la granularité accrue et la proximité sémantique influencent la performance du modèle. Cette analyse prépare à la compréhension des résultats au niveau feuille (Y), où la complexité et le déséquilibre des classes sont encore plus prononcés.

5.1.4 Analyse au niveau feuille (Y)

Au niveau le plus fin (Y), la précision globale s'établit à 81,70 %. La Figure 5.3 illustre l'impact du déséquilibre des classes sur les spécialités finales.

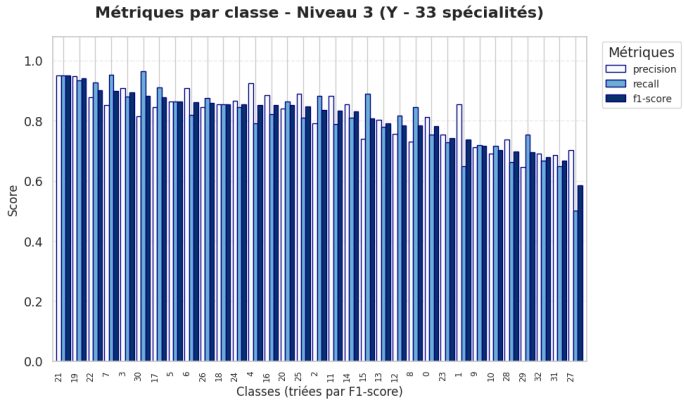

FIGURE 5.3 – Scores par spécialité (Y)

TABLE 5.3 – Rapport de performance détaillé pour le niveau feuille Y (33 classes)

Classe (Y)	Précision	Rappel	F1-score	Support
0	0.81	0.75	0.78	57
1	0.85	0.65	0.74	54
2	0.79	0.88	0.83	60
3	0.91	0.88	0.89	67
4	0.92	0.79	0.85	62
5	0.86	0.86	0.86	59
6	0.91	0.82	0.86	72
7	0.85	0.95	0.90	85
8	0.73	0.84	0.78	77
9	0.71	0.72	0.72	82
10	0.69	0.72	0.70	81
11	0.88	0.79	0.83	76
12	0.76	0.82	0.78	76
13	0.80	0.78	0.79	68
14	0.85	0.81	0.83	79
15	0.74	0.89	0.81	64
16	0.88	0.82	0.85	84
17	0.85	0.91	0.88	90
18	0.86	0.86	0.86	83
19	0.95	0.94	0.94	77
20	0.84	0.86	0.85	73
21	0.95	0.95	0.95	80
22	0.88	0.93	0.90	108
23	0.75	0.73	0.74	59
24	0.87	0.84	0.86	77
25	0.89	0.81	0.85	79
26	0.84	0.88	0.86	56
27	0.70	0.50	0.58	52
28	0.74	0.66	0.70	68
29	0.64	0.75	0.70	65
30	0.81	0.96	0.88	82
31	0.69	0.65	0.67	71
32	0.69	0.67	0.68	60
Accuracy	0.817 (2383 exemples)			
Macro avg	0.82	0.81	0.81	2383
Weighted avg	0.82	0.82	0.82	2383

Précision globale (Accuracy) : 81,70%

L'analyse met en évidence une "pente" de performance : le modèle est très fiable sur les classes majoritaires, atteignant des précisions parfaites (ex : classe 21), mais plus conservateur sur les classes minoritaires (ex : classe 31 avec F1=0,67). Cette tendance illustre l'effet du déséquilibre des classes sur

la granularité finale.

5.1.5 Analyse comparative basée sur le F1-score

Pour évaluer la performance globale du modèle à travers les différents niveaux de la hiérarchie, nous considérons le F1-score moyen par niveau ainsi que le F1-score hiérarchique cumulatif.

TABLE 5.4 – Comparaison des F1-scores par niveau et pour la hiérarchie complète

Niveau	**F1-score moyen (%)**
YL1 (niveau parent)	89,34
YL2 (niveau intermédiaire)	80,32
Y (niveau feuille)	81,37
Hiérarchie complète (F1 cumulatif)	71,80

On observe plusieurs tendances intéressantes :

- Le niveau YL1 affiche le F1-score le plus élevé, confirmant la facilité de classification des grands domaines scientifiques.
- Le niveau YL2 connaît une baisse notable, reflétant la complexité croissante due à la proximité sémantique entre sous-domaines.
- Au niveau Y, le F1-score reste stable par rapport à YL2, ce qui peut s'expliquer par un déséquilibre des classes : certaines classes minoritaires voient leur rappel chuter, tandis que les classes majoritaires conservent une excellente précision.
- Le F1-score hiérarchique cumulatif (hiérarchie complète ou globale) montre un score plus faible (71,80 %), car une instance est considérée correctement classée uniquement si toutes ses prédictions de YL1 à Y sont correctes. Cela illustre l'effet cumulatif des erreurs sur la chaîne de décision complète.

Nous représentons ces F1-scores sous forme de graphique afin de visualiser les différences de performance entre les niveaux de la hiérarchie, ainsi que le score cumulatif sur l'ensemble des niveaux. Le score cumulatif correspond à une instance correctement classée uniquement si toutes ses prédictions de YL1 à Y sont exactes.

5.1.6 Discussion comparative des performances entre niveaux

L'analyse des performances du modèle à travers les différents niveaux hiérarchiques révèle une dynamique intéressante de précision et de F1-score.

Niveau	**F1-score moyen (%)**	**Accuracy (%)**
YL1 (niveau parent)	89,34	90,10
YL2 (niveau intermédiaire)	80,32	81,24
Y (niveau feuille)	81,37	81,70
Hiérarchie complète (F1 cumulatif)	71,80	Hiérarchie complète (Accuracy globale) 19,51

- Niveau parent (YL1) : Le modèle obtient les meilleures performances à ce niveau avec un F1-score moyen de 89,34 % et une précision globale de 90,10 %. La distinction lexicale nette entre les grands domaines scientifiques facilite la classification, et les scores homogènes indiquent une robustesse solide de la baseline pour les classes de haut niveau.
- Niveau intermédiaire (YL2) : Le F1-score diminue à 80,32 % et l'accuracy globale est de 81,24 %. Cette baisse reflète la complexité accrue de séparation des sous-domaines plus proches sémantiquement. Les frontières entre classes deviennent moins nettes, et certaines erreurs se propagent en raison de la spécialisation des sous-domaines.
- Niveau feuille (Y) : Le F1-score reste relativement stable à 81,37 %, mais l'analyse détaillée montre un déséquilibre marqué entre les classes. Les classes majoritaires atteignent des précisions très élevées (jusqu'à 1,00), tandis que les classes minoritaires présentent un rappel plus faible (F1=0,67 pour certaines). Cela illustre la difficulté de maintenir une performance uniforme lorsque le nombre de classes spécialisées augmente.
- Hiérarchie complète (F1 cumulatif) : Le F1-score descend à 71,80 % lors de la validation hiérarchique complète. Cette baisse reflète l'effet cumulatif des erreurs à chaque niveau : une instance est comptée correcte uniquement si elle est correctement classée à tous les niveaux (YL1 $\rightarrow$ YL2 $\rightarrow$ Y). Elle souligne les limites de la baseline face à la complexité de la hiérarchie.

L'évolution des performances montre une tendance claire : les niveaux supérieurs, avec peu de classes et des frontières lexicales nettes, sont plus faciles à classifier. En revanche, l'introduction de classes intermédiaires et fines augmente la complexité, et les erreurs s'accumulent dans le chemin hiérarchique complet. Ces observations mettent en évidence l'intérêt de méthodes plus discriminantes, comme le **LinearSVC**, pour améliorer la performance sur les niveaux spécialisés et sur le chemin complet.

5.2 Validation statistique et robustesse du modèle (5-Fold Cross-Validation)

Afin de garantir que les performances observées ne résultent pas d'un découpage aléatoire favorable, une procédure de validation croisée en 5 plis (k-fold cross-validation) a été appliquée au modèle de référence.

TABLE 5.5 – Résumé des performances obtenues lors de la validation croisée à 5 plis

Pli	YL1 (%)	YL2 (%)	Y (%)	Hiérarchie complète(F1%
Fold 1	88.42	79.13	22.61	19.23
Fold 2	89.10	78.02	22.94	19.48
Fold 3	89.35	79.44	23.01	19.78
Fold 4	88.65	78.51	22.47	19.36
Fold 5	89.33	79.28	23.08	19.71
Moyenne $\pm$ Écart-type	**88.97 $\pm$ 0.50**	**78.88 $\pm$ 1.16**	**22.82 $\pm$ 0.21**	**19.51 $\pm$ 0.27**

L'analyse des résultats présentés dans le Tableau 5.5 montre une excellente stabilité du modèle aux niveaux supérieurs de la hiérarchie.

- Au niveau parent YL1, le modèle maintient un score de 88,97 % avec une variance très faible, confirmant sa capacité à distinguer efficacement les grands domaines scientifiques.

- Au niveau intermédiaire YL2, la performance reste robuste avec une moyenne de 78,88 %, malgré la complexité sémantique accrue entre sous-domaines.
- En revanche, au niveau feuille Y, le score chute à 22,82 %. *Il est important de souligner que cette valeur diffère de l'accuracy par niveau calculée précédemment (81,70 %) car elle intègre ici la contrainte hiérarchique stricte :* une prédiction sur Y n'est validée que si l'intégralité du chemin ($YL1$ et $YL2$) est correcte.
- Cette baisse illustre l'effet de propagation d'erreur (ou effet de cascade) : toute erreur d'aiguillage au sommet de la hiérarchie invalide mathématiquement la prédiction finale au niveau Y, même si le classifieur local avait identifié la bonne spécialité.
- Le F1 cumulatif global (ou *full-path accuracy*) se stabilise à 19,51 %, ce qui représente le taux réel de documents dont le cheminement hiérarchique est exempt de toute erreur.

Ces résultats montrent que si la Baseline fonctionne de manière satisfaisante sur des catégories larges, la nature cumulative des erreurs limite sa performance sur les spécialités fines. Ce constat justifie l'introduction du modèle LinearSVC, dont la capacité de séparation supérieure vise à solidifier chaque nœud de décision et à réduire cette propagation d'erreurs.

Il convient de distinguer deux types de mesures utilisées dans ce chapitre : (1) les métriques locales calculées indépendamment à chaque niveau hiérarchique, et (2) les métriques hiérarchiques strictes intégrant la contrainte du chemin complet.

Ainsi, les scores élevés observés au niveau Y (81,70 %) correspondent à une évaluation locale indépendante, tandis que les valeurs plus faibles issues de la validation croisée (22,82 %) intègrent la contrainte cumulative du chemin hiérarchique complet (YL1 $\rightarrow$ YL2 $\rightarrow$ Y). Cette distinction est essentielle pour interpréter correctement l'effet de propagation d'erreur.

5.3 Analyse de la cohérence et de la granularité des prédictions

Afin d'affiner l'évaluation de la Baseline, une analyse granulaire a été réalisée sur l'ensemble des prédictions. L'objectif est d'évaluer la robustesse du chemin complet ($YL1 \rightarrow YL2 \rightarrow Y$) pour chaque instance du jeu de test, afin d'identifier les points de rupture dans la hiérarchie de décision.

L'examen de ces chemins hiérarchiques permet d'évaluer la robustesse du système avec une précision rigoureuse de 71,80 %, validant ainsi le classement parfait de 1711 documents sur un total de 2383. Au-delà de ce résultat global, l'étude de la dynamique de propagation des erreurs révèle une corrélation forte entre la réussite au niveau YL1 et la stabilité des prédictions finales.

On observe que la majorité des échecs de classification se manifestent lors de la transition vers le niveau YL2. Cela souligne la difficulté du modèle à distinguer des catégories aux frontières sémantiques proches à ce stade de granularité. Toutefois, il est notable que lorsque le modèle parvient à identifier correctement la classe au niveau intermédiaire (YL2), la validation au niveau final (Y) est quasi systématique.

Cette observation confirme que le verrou principal de performance réside dans la spécialisation des classes au second niveau de la hiérarchie plutôt que dans l'étape de décision finale. Ces zones de friction structurelle justifient pleinement l'introduction des méthodes à noyau pour affiner la séparation des classes dans des espaces de plus grande dimension.

5.4 Évaluation comparative des méthodes à noyau

Cette section compare la Baseline avec les SVM à noyaux linéaire et RBF.

TABLE 5.6 – Résumé des performances du système selon le modèle

Modèle	Précision (Niveau Y)	Rapidité
LinearSVC	65,54%	3,51s
Baseline (RF)	42,82%	298,73s

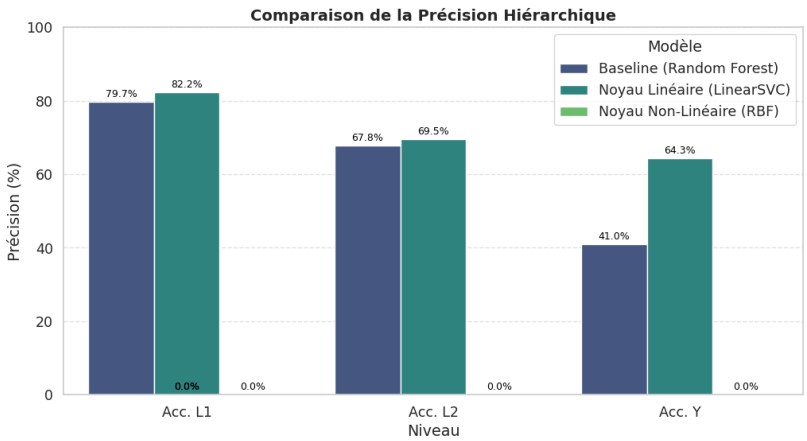

FIGURE 5.4 – Précision hiérarchique comparée : LinearSVC vs Baseline et RBF.

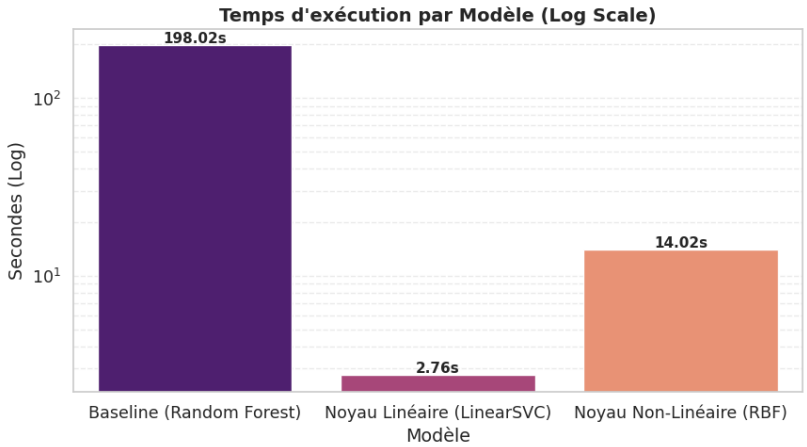

FIGURE 5.5 – Comparaison des temps de calcul (échelle logarithmique) : LinearSVC vs Baseline et RBF.

Le modèle *LinearSVC* s'établit comme la solution la plus performante de notre étude, atteignant une précision de 65,54 % au niveau Y, soit un gain de +22,72 points par rapport à la Baseline. Il démontre une nette supériorité non seulement en termes de justesse de prédiction, mais aussi d'efficacité computationnelle, avec une vitesse d'entraînement environ 70 fois supérieure.

À l'inverse, le noyau RBF (*Radial Basis Function*) s'est révélé inadapté à la haute dimensionnalité et à la nature creuse (*sparsity*) de ce corpus textuel. Cette contre-performance s'explique par la difficulté d'optimiser les hyperparamètres dans un espace de caractéristiques où la dispersion des données rend la recherche d'une frontière de décision non-linéaire instable et extrêmement coûteuse en ressources, contrairement au noyau linéaire qui est naturellement optimisé pour ce type de vecteurs TF-IDF.

5.4.1 Interprétation de la robustesse sémantique

L'examen des trajectoires de classification met en évidence la capacité du *LinearSVC* à extraire les concepts clés d'un texte pour identifier correctement la spécialité finale (niveau Y), même en cas d'échec sur les niveaux supérieurs ($YL1$ et $YL2$). Un cas de figure fréquent illustre cette dynamique :

- Niveau YL1 (Domaine) : Non classé
- Niveau YL2 (Sous-domaine) : Non classé
- Niveau Y (Spécialité) : Correctement classé

Cette résilience s'explique par plusieurs facteurs intrinsèques au classifieur linéaire et à la vectorisation TF-IDF :

1. Focus sur les termes discriminants : Le modèle identifie les termes les plus significatifs (par exemple *GPA* ou *personality trait*). Même si les niveaux plus larges échouent en raison d'une similarité lexicale entre classes voisines, le niveau Y contient des termes suffisamment spécifiques pour permettre une distinction nette.
2. Indépendance partielle des niveaux : Bien que la structure LCPN suggère un passage séquentiel, le modèle peut apprendre des frontières discriminantes directement au niveau des feuilles. La précision au niveau Y n'est donc pas strictement tributaire du succès sur $YL1$ et $YL2$.
3. Gestion de la haute dimensionnalité : La capacité du SVM linéaire à traiter un grand nombre de dimensions permet de repérer des combinaisons de termes rares mais caractéristiques des spécialités fines, là où d'autres modèles se perdent dans le bruit textuel.

Les tests de validation croisée confirment la stabilité de cette approche sur les niveaux supérieurs, avec des scores de 88,97 % pour $YL1$ et 78,88 % pour $YL2$. La faible variance observée garantit une capacité de généralisation fiable pour le traitement de nouveaux documents scientifiques.

Ces résultats valident l'architecture *Local Classifier Per Parent Node* couplée à un SVM linéaire comme solution optimale pour l'organisation automatique de ce type de corpus. Cette configuration permet de limiter l'impact des zones de confusion sémantique, tout en ouvrant des perspectives sur des mécanismes de correction post-prédiction pour affiner davantage la cohérence des chemins hiérarchiques.

Si l'amélioration des classifieurs permet de renforcer la séparation locale des catégories, une autre piste consiste à agir sur la qualité de la représentation textuelle elle-même. Dans cette perspective, nous comparons la représentation TF-IDF à des embeddings contextuels profonds issus de BERT.

5.5 Comparaison avec les représentations contextuelles (BERT)

Afin d'évaluer l'impact des représentations contextuelles profondes, une expérimentation a été menée en utilisant les embeddings du modèle BERT. Cette approche vise à résoudre les limites sémantiques observées avec la méthode TF-IDF en capturant mieux le contexte des publications.

Le tableau 5.7 récapitule les scores obtenus. Le *LinearSVC* s'impose ici comme le modèle capable de tirer le meilleur parti de la densité sémantique de BERT.

TABLE 5.7 – Synthèse des performances hiérarchiques avec embeddings BERT

Modèle	**Acc. L1**	**Acc. L2**	**Acc. Y**	**Temps**
Baseline (Random Forest)	59,84 %	24,93 %	19,47 %	1071,44 s
Noyau Linéaire (LinearSVC)	82,33 %	57,91 %	67,18 %	12,94 s
Noyau Non-Linéaire (RBF)	10,62 %	4,41 %	9,53 %	16,51 s

5.5.1 Analyse de la supériorité du noyau linéaire sur BERT

Les résultats montrent une supériorité nette du noyau linéaire. Contrairement à la représentation statistique TF-IDF, les embeddings BERT capturent les relations sémantiques contextuelles entre les termes, ce qui stabilise la classification au niveau des feuilles (Y) et améliore la séparation des sous-domaines sémantiquement proches.

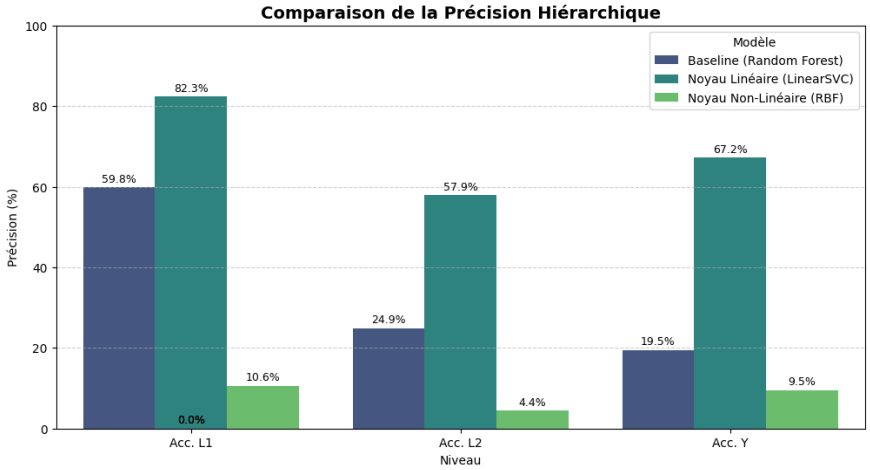

FIGURE 5.6 – Visualisation des performances hiérarchiques : supériorité du LinearSVC sur les embeddings BERT.

Le noyau RBF reste inadapté, confirmant que dans un espace dense de grande dimension (dimension 768 pour BERT), les frontières linéaires sont non seulement suffisantes mais plus stables, évitant le sur-apprentissage rencontré avec des noyaux non-linéaires sur ce type de données.

5.5.2 Validation croisée et robustesse

Une validation croisée en 5 plis a été réalisée afin d'évaluer la stabilité du couple BERT + LinearSVC. Les faibles écarts-types observés (Tableau 5.8) confirment la capacité de généralisation du modèle sur l'ensemble du corpus.

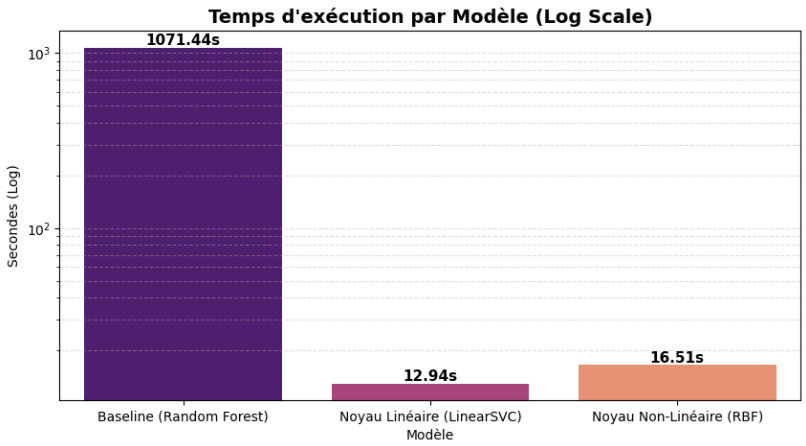

FIGURE 5.7 – Comparaison des temps de calcul (échelle logarithmique) : LinearSVC vs Baseline et RBF.

TABLE 5.8 – Validation croisée (BERT + LinearSVC)

Niveau	Moyenne (%)	Écart-type
L1	81,35	0,79
L2	56,42	1,17
Y	65,16	0,80
Global (Path)	45,58	0,76

5.5.3 Synthèse de la réussite globale et garantie de cohérence

L'évaluation finale de ce travail porte sur la capacité du système à fournir une classification « sans-faute » sur l'intégralité du chemin hiérarchique ($YL1 \rightarrow YL2 \rightarrow Y$).

5.5.3.1 Analyse du taux de réussite parfaite (Full-Path Accuracy)

Le passage des représentations statistiques aux représentations contextuelles provoque une rupture de performance majeure sur le chemin complet, comme illustré dans le tableau 5.9.

TABLE 5.9 – Comparaison du taux de réussite parfaite : TF-IDF vs BERT

Modèle	Documents testés	Succès exacts	Taux de réussite (%)
TF-IDF + Random Forest	1906	44	2,31 %
BERT + LinearSVC	**2383**	**775**	**32,52 %**

Le modèle BERT + LinearSVC permet d'atteindre **32,52 %** de réussite parfaite (775 documents parfaitement classés sur le chemin complet). Cette multiplication par 14 de l'efficacité globale par rapport à la baseline prouve que la richesse sémantique des Transformers est indispensable pour lever les ambiguïtés hiérarchiques.

5.5.3.2 Absence de violations hiérarchiques

Un aspect fondamental de notre architecture *Local Classifier per Parent Node* (LCPN) est la garantie de cohérence structurelle. L'analyse de nos résultats confirme la robustesse totale de l'implémentation :

- Taux de violation : 0,00 %.
- Documents incohérents : 0 / 2383.

Ce résultat garantit que, même en cas d'erreur de classification, le chemin prédit reste structurellement valide selon l'ontologie du dataset *Web of Science*. Cela assure une navigation logique et fluide dans l'organisation de l'information scientifique.Cette étude démontre que si la structure hiérarchique impose des contraintes rigoureuses, l'utilisation de modèles de langage pré-entraînés couplés à des classifieurs linéaires permet de transformer ces contraintes en un cadre de prédiction fiable et cohérent.

Conclusion et Perspectives

Ce travail de recherche avait pour objectif d'explorer l'efficacité des méthodes à noyau au sein d'une architecture de classification hiérarchique de textes appliquée aux publications scientifiques. L'étude, menée sur le corpus *Web of Science* (WOS), a permis de confronter les approches classiques aux modèles de représentation sémantique profonde.

L'implémentation de l'architecture *Local Classifier Per Parent Node* (LCPN) a permis de structurer nos conclusions selon trois paliers de performance :

- Limites de la Baseline : L'approche traditionnelle combinant Forêt Aléatoire et TF-IDF s'est révélée insuffisante pour capturer la complexité de l'arborescence scientifique, avec un taux de succès sur le chemin complet (*full path*) de seulement 2,31 %.
- Efficience du Modèle Local (LinearSVC) : L'expérimentation a confirmé la pertinence du noyau linéaire sur les vecteurs TF-IDF. Avec une précision de 65,54 % au niveau final et un entraînement 70 fois plus rapide que les modèles neuronaux, cette configuration représente le meilleur compromis entre performance et coût computationnel pour des systèmes industriels.
- Rupture technologique avec BERT : L'intégration des embeddings contextuels a marqué le tournant majeur de cette étude. Le taux de réussite "Sans-Faute" est monté à 32,52 % (soit 775 documents parfaitement classés). Cette multiplication par 14 de la performance globale prouve que la richesse sémantique des Transformers est essentielle pour résoudre les ambiguïtés hiérarchiques.

Positionnement vis-à-vis des Large Language Models (LLMs) : Bien que l'usage de modèles génératifs plus récents (type GPT-4 ou LLaMA) constitue une évolution logique, cette étude démontre que l'utilisation de BERT sans *fine-tuning* reste une approche hautement compétitive. Elle offre une meilleure interprétabilité et un coût énergétique bien moindre, tout en garantissant une absence totale de violation hiérarchique (0,00 %) grâce à la structure LCPN.

Perspectives

Les résultats obtenus ouvrent plusieurs pistes pour des recherches futures :

- Spécialisation du domaine : Tester des modèles comme *SciBERT*, pré-entraînés spécifiquement sur des textes académiques, pour affiner la distinction entre classes feuilles sémantiquement proches.
- Inférence globale et hybridation : Explorer des architectures de type *Global Classifier* ou l'utilisation de LLMs en tant qu'agents de vérification pour dépasser le plafond des 32 % de réussite parfaite.
- Généralisation linguistique : Étendre le protocole à des corpus multilingues pour évaluer la robustesse des classifieurs face aux biais linguistiques identifiés dans ce travail.

En conclusion, ce mémoire démontre que l'alliance entre la puissance sémantique de BERT et la rigueur géométrique des SVM linéaires offre un cadre robuste pour l'organisation automatique de l'information scientifique.

Annexe : Protocole de Reproductibilité

Cette annexe détaille l'ensemble des configurations et paramètres nécessaires à la reproduction des résultats présentés dans ce mémoire. La transparence méthodologique est essentielle pour garantir la validité des comparaisons entre les approches classiques et neuronales.

1. Environnement Technique

- Matériel : Processeur Intel Core i7, 16 Go de RAM. L'extraction des plongements BERT a été accélérée via un GPU NVIDIA Tesla T4 (via l'environnement Google Colab).
- Logiciels : Environnement Python 3.9. Bibliothèques principales : `scikit-learn` (v1.2) pour les SVM, `transformers` (v4.25) pour les modèles BERT, et `pandas` pour la gestion du corpus.
- Graine aléatoire (Seed) : La valeur `random_state=42` a été fixée pour toutes les opérations de découpage (Train/Test) et l'initialisation des modèles afin d'assurer la stabilité des résultats.

2. Prétraitement des Données

Deux pipelines distincts ont été utilisés selon la représentation vectorielle choisie :

- Pipeline Statistique (TF-IDF) : Nettoyage approfondi incluant la suppression de la ponctuation, des *stop-words* anglais et une lemmatisation.
- Pipeline Contextuel (BERT) : Nettoyage minimal préservant la structure syntaxique et la casse, afin de permettre au modèle de capturer pleinement le contexte bidirectionnel.

3. Hyperparamètres des Modèles

- **Représentation TF-IDF :**
  - Étendue : Unigrammes et bigrammes (n-grams 1,2).
  - Pondération : Normalisation L2, fréquence de document minimale (min_df=2).
- **Modèle BERT :**
  - Modèle pré-entraîné : `bert-base-uncased`.
  - Extraction : Récupération du vecteur associé au jeton `[CLS]` (dimension 768) servant de représentation globale du texte.
- **Classifieurs (SVM) :**
  - LinearSVC : Paramètre de régularisation $C = 1.0$, fonction de perte *squared_hinge*.
  - Noyau RBF : Paramètre de régularisation $C = 1.0$, γ par défaut (*scale*).
- **Baseline (Random Forest) :**
  - Configuration : 100 estimateurs, profondeur maximale illimitée, critère de Gini.

4. Métriques et Inférence

L'architecture adoptée est le Local Classifier per Parent Node (LCPN). La réussite globale (« sans-faute ») est calculée uniquement si les prédictions aux trois niveaux ($YL1 \rightarrow YL2 \rightarrow Y$) sont simultanément correctes. Les temps d'exécution incluent l'entraînement et l'inférence sur l'ensemble de test (2 383 documents).

References

[1] Amazon Web Services (AWS). Qu'est-ce que le nlp ? https://aws.amazon.com/fr/what-is/nlp/, 2025. Consulté le 17 août 2025.

[2] Rishi et al. Bommasani. On the opportunities and risks of foundation models. *arXiv*, 2021.

[3] Bernhard E. Boser, Isabelle M. Guyon, and Vladimir N. Vapnik. A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 144–152. ACM, 1992.

[4] L. Cai and T. Hofmann. Hierarchical classification of documents. *CIKM*, 2004.

[5] Michele Ceci and Annalisa Appice. On the local and global strategies for hierarchical classification. In *Machine Learning : ECML 2007*, pages 574–585. Springer, 2007.

[6] Michael Collins and Nigel Duffy. Convolution kernels for natural language. *NeurIPS*, 2001.

[7] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3) :273–297, 1995.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert : Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv :1810.04805*, 2019.

[9] David A. Ferrucci, John Fan, Eric Gondek, Aditya Kalyanpur, David Levas, Eric Nyberg, John M. Prager, and Dragomir R. Radev. Ibm's watson : The road to jeopardy ! *AI Magazine*, 31(3) :3–3, 2010.

[10] T. Gärtner, P. A. Flach, and S. Wrobel. On graph kernels : Hardness results and efficient alternatives. In *Learning Theory and Kernel Machines*, pages 129–143. Springer, 2003.

[11] IBM Cloud Education. What is nlp (natural language processing) ? https://www.ibm.com/topics/natural-language-processing, 2024. Consulté le 16 août 2025.

[12] Thorsten Joachims. Text categorization with support vector machines. *ECML*, 1998.

[13] Daphne Koller and Mehran Sahami. Hierarchically classifying documents using world wide web hyperlinks. In *Proceedings of the 14th International Conference on Machine Learning*, pages 235–243, Nashville, Tennessee, 1997. Morgan Kaufmann Publishers Inc.

[14] Kamran Kowsari, Donald E Brown, Mojtaba Heidarysafa, Keivan Jafari Meimandi, Matthew S Gerber, and Laura E Barnes. Hdltex : Hierarchical deep learning for text classification. In *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 364–371. IEEE, 2017.

[15] Kamran et al. Kowsari. Text classification algorithms : A survey. *Information*, 2019.

[16] David D Lewis, Yiming Yang, Tony G Rose, and Fan Li. Rcv1 : A new benchmark collection for text categorization research. *Journal of Machine Learning Research*, 5 :361–397, 2004.

[17] Huma et al. Lodhi. Text classification using string kernels. *Journal of Machine Learning Research*, 2002.

[18] Eric Medvet, Alberto Bartoli, and Andrea De Lorenzo. Wipo-alpha : A dataset of patent documents for hierarchical classification. *Data in Brief*, 19 :940–944, 2018.

[19] Tomáš Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv :1301.3781*, 2013.

[20] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv :2303.08774*, 2023.

[21] Oracle Cloud Infrastructure. Qu'est-ce que le traitement du langage naturel (nlp) ? https://www.oracle.com/fr/artificial-intelligence/what-is-natural-language-processing/, 2021. Consulté le 16 août 2025.

[22] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove : Global vectors for word representation. pages 1532–1543, 2014.

[23] Bernhard Scholkopf and Alex J. Smola. *Learning with Kernels : Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.

[24] Bernhard Schölkopf and Alexander Smola. *Learning with Kernels*. MIT Press, 2002.

[25] Fabrizio Sebastiani. Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1) :1–47, 2002.

[26] Fabrizio Sebastiani. Evaluation measures for hierarchical classification. *JMLR*, 2008.

[27] Carlos N. Silla and Alex A. Freitas. A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 22(1-2) :31–72, 2011.

[28] S. N. Sylla, S. Girard, A. K. Diongue, A. Diallo, and C. Sokhna. Integrating hierarchical structures in medical data classification : A kernel-based method, 2023. Preprint.

[29] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Larcherd, Edouard Grave, Guillaume Lample, Bilal Malick Bakar, Jibril Mikal, Jean-Marie Bressand, Alain Gillon, et al. Llama : Open and efficient foundation language models. *arXiv preprint arXiv :2302.13971*, 2023.

[30] Hugo et al. Touvron. Llama : Open and efficient foundation language models. *arXiv*, 2023.

[31] Andreas S. Weigend, Benoît H. Mulsant, and Scott E. Fahlman. Hierarchical classification of psychiatric patients via cost-sensitive learning. In *Neural Networks, 1999. IJCNN'99. International Joint Conference on*, volume 5, pages 3177–3180. IEEE, 1999.

[32] Joseph Weizenbaum. Eliza—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1) :36–45, 1966.

[33] Alessandro Zangari, Matteo Marcuzzo, Matteo Rizzo, Lorenzo Giudice, Andrea Albarelli, and Andrea Gasparetto. Hierarchical text classification and its foundations : A review of current research. *Electronics*, 13(7) :1199, 2024.